Article

Champion-level drone racing using deep reinforcement learning

https://doi.org/10.1038/s41586-023-06419-4

Received: 5 January 2023

Accepted: 10 July 2023

Published online: 30 August 2023

Open access

Check for updates

Elia Kaufmann[1]✉, Leonard Bauersfeld[1], Antonio Loquercio[1], Matthias Müller[2], Vladlen Koltun[3] & Davide Scaramuzza[1]

First-person view (FPV) drone racing is a televised sport in which professional competitors pilot high-speed aircraft through a 3D circuit. Each pilot sees the environment from the perspective of their drone by means of video streamed from an onboard camera. Reaching the level of professional pilots with an autonomous drone is challenging because the robot needs to fly at its physical limits while estimating its speed and location in the circuit exclusively from onboard sensors[1]. Here we introduce Swift, an autonomous system that can race physical vehicles at the level of the human world champions. The system combines deep reinforcement learning (RL) in simulation with data collected in the physical world. Swift competed against three human champions, including the world champions of two international leagues, in real-world head-to-head races. Swift won several races against each of the human champions and demonstrated the fastest recorded race time. This work represents a milestone for mobile robotics and machine intelligence[2], which may inspire the deployment of hybrid learning-based solutions in other physical systems.

Deep RL[3] has enabled some recent advances in artificial intelligence. Policies trained with deep RL have outperformed humans in complex competitive games, including Atari[4–6], Go[5,7–9], chess[5,9], StarCraft[10], Dota 2 (ref. 11) and Gran Turismo[12,13]. These impressive demonstrations of the capabilities of machine intelligence have primarily been limited to simulation and board-game environments, which support policy search in an exact replica of the testing conditions. Overcoming this limitation and demonstrating champion-level performance in physical competitions is a long-standing problem in autonomous mobile robotics and artificial intelligence[14–16].

FPV drone racing is a televised sport in which highly trained human pilots push aerial vehicles to their physical limits in high-speed agile manoeuvres (Fig. 1a). The vehicles used in FPV racing are quadcopters, which are among the most agile machines ever built (Fig. 1b). During a race, the vehicles exert forces that surpass their own weight by a factor of five or more, reaching speeds of more than 100 km h^{-1} and accelerations several times that of gravity, even in confined spaces. Each vehicle is remotely controlled by a human pilot who wears a headset showing a video stream from an onboard camera, creating an immersive 'first-person-view' experience (Fig. 1c).

Attempts to create autonomous systems that reach the performance of human pilots date back to the first autonomous drone racing competition in 2016 (ref. 17). A series of innovations followed, including the use of deep networks to identify the next gate location[18–20], transfer of racing policies from simulation to reality[21,22] and accounting for uncertainty in perception[23,24]. The 2019 AlphaPilot autonomous drone racing competition showcased some of the best research in the field[25]. However, the first two teams still took almost twice as long as a professional human pilot to complete the track[26,27]. More recently, autonomous systems have begun to reach expert human performance[28–30]. However, these works rely on near-perfect state estimation provided by an external motion-capture system. This makes the comparison with human pilots unfair, as humans only have access to onboard observations from the drone.

In this article, we describe Swift, an autonomous system that can race a quadrotor at the level of human world champions using only onboard sensors and computation. Swift consists of two key modules: (1) a perception system that translates high-dimensional visual and inertial information into a low-dimensional representation and (2) a control policy that ingests the low-dimensional representation produced by the perception system and produces control commands.

The control policy is represented by a feedforward neural network and is trained in simulation using model-free on-policy deep RL[31]. To bridge discrepancies in sensing and dynamics between simulation and the physical world, we make use of non-parametric empirical noise models estimated from data collected on the physical system. These empirical noise models have proved to be instrumental for successful transfer of the control policy from simulation to reality.

We evaluate Swift on a physical track designed by a professional drone-racing pilot (Fig. 1a). The track comprises seven square gates arranged in a volume of 30 × 30 × 8 m, forming a lap of 75 m in length. Swift raced this track against three human champions: Alex Vanover, the 2019 Drone Racing League world champion, Thomas Bitmatta, two-time MultiGP International Open World Cup champion, and Marvin Schaepper, three-time Swiss national champion. The quadrotors used by Swift and by the human pilots have the same weight, shape and propulsion. They are similar to drones used in international competitions.

The human pilots were given one week of practice on the race track. After this week of practice, each pilot competed against Swift in several head-to-head races (Fig. 1a,b). In each head-to-head race, two drones

[1]Robotics and Perception Group, University of Zurich, Zürich, Switzerland. [2]Intel Labs, Munich, Germany. [3]Intel Labs, Jackson, WY, USA. ✉e-mail: ekaufmann@ifi.uzh.ch

文章

使用深度强化学习的冠军级无人机竞赛

https://doi.org/10.1038/s41586-023-06419-4

收稿日期：2023 年 1 月 5 日

接受日期：2023 年 7 月 10 日

在线发布：2023 年 8 月 30 日

开放获取

检查更新

埃利亚·考夫曼[1]、伦纳德·鲍尔斯菲尔德[1]、安东尼奥·洛奎西奥[1]、马蒂亚斯·穆勒[2]、弗拉德伦·科尔通[3]&达维德·斯卡拉穆扎[1]

第一人称视角 (FPV) 无人机竞赛是一项电视转播的运动，专业选手在 3D 赛道上驾驶高速飞机。每个飞行员通过从无人机传输的视频从无人机的角度观察环境

机载摄像头。使用自主无人机达到专业飞行员的水平具有挑战性，因为机器人需要在其物理极限下飞行，同时仅通过机载传感器估计其速度和在电路中的位置[1]。在这里，我们介绍 Swift，这是一个能够以人类世界冠军水平对物理车辆进行比赛的自主系统。该系统结合了深度强化学习（RL）

使用在物理世界中收集的数据进行模拟。斯威夫特在现实世界的面对面比赛中与三名人类冠军竞争，其中包括两个国际联盟的世界冠军。斯威夫特在与每位人类冠军的比赛中赢得了几场比赛，并展示了最快的比赛时间记录。这项工作代表了移动机器人和机器智能[2]的里程碑，可能会激发在其他物理系统中部署基于混合学习的解决方案。

深度 RL[3] 推动了人工智能领域的一些最新进展。 经过深度强化学习训练的策略在复杂的竞技游戏中表现优于人类，包括 Atari[4–6]、Go[5,7–9]、国际象棋[5,9]、StarCraft[10]、Dota 2 (ref. 11) 和 Gran Turismo[12,13]。这些令人印象深刻的机器智能功能演示主要局限于模拟和棋盘游戏环境，这些环境支持在测试条件的精确复制中进行策略搜索。克服这一限制并在体育比赛中展示冠军级表现是自主移动机器人和人工智能领域长期存在的问题[14–16]。

FPV 无人机竞赛是一项电视转播的运动，训练有素的人类飞行员在高速敏捷机动中将飞行器推向物理极限（图 1a）。 FPV 赛车中使用的车辆是四轴飞行器，它是有史以来最敏捷的机器之一（图 1b）。在比赛中，车辆施加的力超过自身重量五倍或更多，即使在有限的空间内，速度也能达到超过 100 km h^{-1} 和几倍于重力的加速度。每辆车都由一名戴着耳机的人类飞行员远程控制，该耳机显示来自车载摄像头的视频流，从而创造出身临其境的“第一人称视角”体验（图 1c）。

创建达到人类飞行员性能的自主系统的尝试可以追溯到 2016 年第一届自主无人机竞赛（参考文献 17）。随后出现了一系列创新，包括使用深度网络来识别下一个登机口位置[18–20]、将赛车政策从模拟转移到现实[21,22]以及考虑感知的不确定性[23,24]。 2019 年 AlphaPilot 自主无人机竞赛展示了该领域的一些最佳研究成果[25]。 然而，前两支队伍完成赛道所需的时间几乎是专业人类飞行员的两倍[26,27]。最近，自主系统已经开始达到人类专家的水平[28–30]。 然而，这些工作依赖于提供的近乎完美的状态估计通过外部动作捕捉系统。这使得与人类飞行员的比较不公平，因为人类只能从无人机上获得机载观察结果。

在本文中，我们描述了 Swift，这是一个仅使用机载传感器和计算就能以人类世界冠军水平进行四旋翼飞行器竞赛的自主系统。 Swift 由两个关键模块组成：（1）感知系统，将高维视觉和惯性信息转换为低维表示；（2）控制策略，摄取感知系统产生的低维表示并产生控制命令。

控制策略由前馈神经网络表示，并使用无模型的同策略深度 RL[31] 进行模拟训练。 为了弥合模拟与物理世界之间传感和动力学方面的差异，我们利用根据物理系统收集的数据估计的非参数经验噪声模型。事实证明，这些经验噪声模型有助于将控制策略从模拟成功转移到现实。

我们在专业无人机竞赛飞行员设计的物理赛道上评估 Swift（图 1a）。赛道由七个方形门组成，排列在 30 × 30 × 8 m 的体积中，形成一圈长度为 75 m 的跑道。 斯威夫特在这条赛道上与三位人类冠军进行了比赛：2019 年无人机竞速联盟世界冠军 Alex Vanover、两届 MultiGP 国际公开赛世界杯冠军 Thomas Bitmatta 和三届瑞士国家冠军 Marvin Schaepper。斯威夫特和人类飞行员使用的四旋翼具有相同的重量、形状和推进力。它们类似于国际比赛中使用的无人机。

人类飞行员在赛道上进行了为期一周的练习。 经过这周的练习后，每位飞行员都与斯威夫特进行了几场正面交锋的比赛（图 1a，b）。在每场对决中，都有两架无人机

[1]Robotics and Perception Group, University of Zurich, Zürich, Switzerland. [2]Intel Labs, Munich, Germany. [3]Intel Labs, Jackson, WY, USA. ✉e-mail: ekaufmann@ifi.uzh.ch

Fig. 1 | Drone racing. a, Swift (blue) races head-to-head against Alex Vanover, the 2019 Drone Racing League world champion (red). The track comprises seven square gates that must be passed in order in each lap. To win a race, a competitor has to complete three consecutive laps before its opponent. **b**, A close-up view of Swift, illuminated with blue LEDs, and a human-piloted drone, illuminated with red LEDs. The autonomous drones used in this work rely only on onboard sensory measurements, with no support from external infrastructure, such as motion-capture systems. **c**, From left to right: Thomas Bitmatta, Marvin Schaepper and Alex Vanover racing their drones through the track. Each pilot wears a headset that shows a video stream transmitted in real time from a camera aboard their aircraft. The headsets provide an immersive 'first-person-view' experience. **c**, Photo by Regina Sablotny.

(one controlled by a human pilot and one controlled by Swift) start from a podium. The race is set off by an acoustic signal. The first vehicle that completes three full laps through the track, passing all gates in the correct order in each lap, wins the race.

Swift won several races against each of the human pilots and achieved the fastest race time recorded during the events. Our work marks the first time, to our knowledge, that an autonomous mobile robot achieved world-champion-level performance in a real-world competitive sport.

The Swift system

Swift uses a combination of learning-based and traditional algorithms to map onboard sensory readings to control commands. This mapping comprises two parts: (1) an observation policy, which distils high-dimensional visual and inertial information into a task-specific low-dimensional encoding, and (2) a control policy that transforms the encoding into commands for the drone. A schematic overview of the system is shown in Fig. 2.

The observation policy consists of a visual–inertial estimator[32,33] that operates together with a gate detector[26], which is a convolutional neural network that detects the racing gates in the onboard images. Detected gates are then used to estimate the global position and orientation of the drone along the race track. This is done by a camera-resectioning algorithm[34] in combination with a map of the track. The estimate of the global pose obtained from the gate detector is then combined with the estimate from the visual–inertial estimator by means of a Kalman filter, resulting in a more accurate representation of the robot's state. The control policy, represented by a two-layer perceptron, maps the output of the Kalman filter to control commands for the aircraft. The policy is trained using on-policy model-free deep RL[31] in simulation. During training, the policy maximizes a reward that combines progress towards the next racing gate[35] with a perception objective that rewards keeping the next gate in the field of view of the camera. Seeing the next gate is rewarded because it increases the accuracy of pose estimation.

Optimizing a policy purely in simulation yields poor performance on physical hardware if the discrepancies between simulation and reality are not mitigated. The discrepancies are caused primarily by two factors: (1) the difference between simulated and real dynamics and (2) the noisy estimation of the robot's state by the observation policy when provided with real sensory data. We mitigate these discrepancies by collecting a small amount of data in the real world and using this data to increase the realism of the simulator.

Specifically, we record onboard sensory observations from the robot together with highly accurate pose estimates from a motion-capture system while the drone is racing through the track. During this data-collection phase, the robot is controlled by a policy trained in simulation that operates on the pose estimates provided by the motion-capture system. The recorded data allow to identify the characteristic failure modes of perception and dynamics observed through the race track. These intricacies of failing perception and unmodelled dynamics are dependent on the environment, platform, track and sensors. The perception and dynamics residuals are modelled using Gaussian processes[36] and k-nearest-neighbour regression, respectively. The motivation behind this choice is that we empirically

图 1 | 无人机竞赛。 a，斯威夫特（蓝色）与 2019 年无人机竞速联盟世界冠军 Alex Vanover（红色）正面交锋。赛道由七个方形大门组成，每一圈都必须按顺序通过。要赢得比赛，参赛者必须连续领先对手完成三圈。 b，用蓝色 LED 照明的 Swift 和用红色 LED 照明的有人驾驶的无人机的特写视图。这项工作中使用的自主无人机仅依赖于机载感官测量，没有外部基础设施（例如动作捕捉系统）的支持。 c，从左到右：Thomas Bitmatta、Marvin Schaepper 和 Alex Vanover 驾驶无人机穿过赛道。每位飞行员都戴着耳机，显示从飞机上的摄像头实时传输的视频流。耳机提供身临其境的“第一人称视角”体验。 c， 摄影：Regina Sablotny。

(一架由人类飞行员控制，一架由 Swift) 控制，从讲台上出发。比赛由声音信号开始。第一辆车在赛道上跑完三圈，每圈都以正确的顺序通过所有大门，赢得比赛。

斯威夫特在与每位人类飞行员的几场比赛中获胜，并取得了赛事期间记录的最快比赛时间。据我们所知，我们的工作标志着自主移动机器人首次在现实世界的竞技运动中取得了世界冠军级的表现。

斯威夫特系统

Swift 结合了基于学习的算法和传统算法，将机载感官读数映射到控制命令。该映射包括两部分：（1）观察策略，将高维视觉和惯性信息提炼为特定于任务的低维编码；（2）控制策略，将编码转换为无人机的命令。该系统的示意图如图 2所示。

观察策略由视觉惯性估计器[32,33]组成，该估计器与门检测器[26]一起运行，门检测器是一个卷积神经网络，用于检测机载图像中的赛车门。然后使用检测到的门来估计无人机沿赛道的全球位置和方向。这是通过相机后方交会算法[34]结合赛道地图来完成的。然后，通过卡尔曼滤波器将从门检测器获得的全局姿态的估计与视觉惯性估计器的估计相结合，从而得到更准确的表示

机器人的状态。控制策略由两层感知器表示，将卡尔曼滤波器的输出映射到飞机的控制命令。该策略是在模拟中使用基于策略的无模型深度 RL[31] 进行训练的。在训练期间，该策略将奖励最大化，该奖励将下一个赛车门[35]的进展与奖励将下一个门保持在摄像机视野内的感知目标相结合。看到下一个门会得到奖励，因为它提高了姿态估计的准确性。

如果不缩小模拟与现实之间的差异，纯粹在模拟中优化策略会导致物理硬件性能较差。这些差异主要由两个因素引起：（1）模拟动态和真实动态之间的差异；（2）在提供真实传感数据时，观察策略对机器人状态的噪声估计。我们通过收集现实世界中的少量数据并使用这些数据来提高模拟器的真实感来减轻这些差异。

具体来说，当无人机在赛道上比赛时，我们会记录机器人的机载感官观察结果以及运动捕捉系统的高精度姿态估计。 在此数据收集阶段，机器人由模拟训练的策略控制，该策略根据运动捕捉系统提供的姿态估计进行操作。记录的数据可以识别通过赛道观察到的感知和动态的特征故障模式。这些错综复杂的感知失败和未建模的动态取决于环境、平台、赛道和传感器。感知和动态残差分别使用高斯过程[36]和k最近邻回归进行建模。这一选择背后的动机是我们根据经验

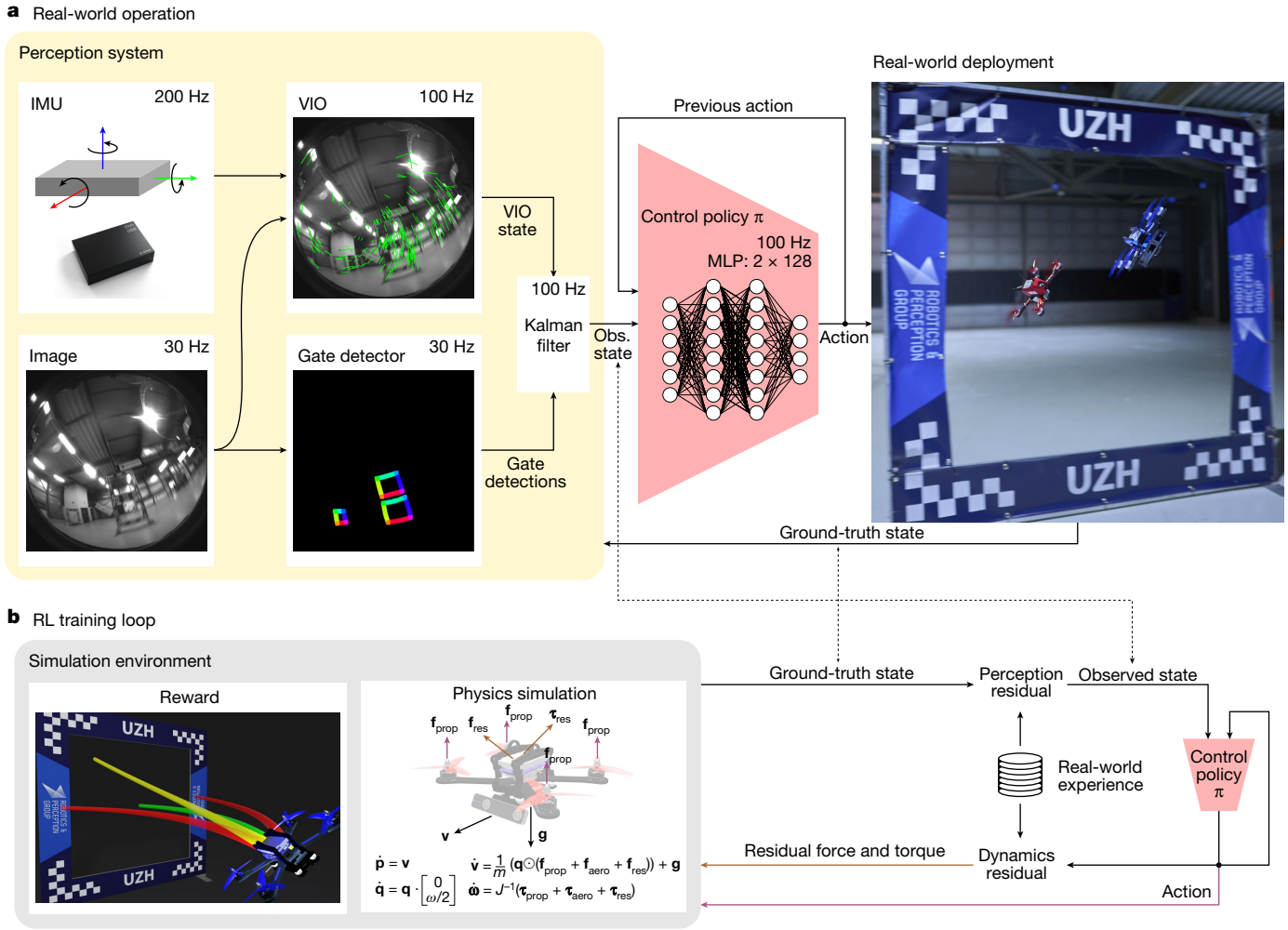


Fig. 2 | The Swift system. Swift consists of two key modules: a perception system that translates visual and inertial information into a low-dimensional state observation and a control policy that maps this state observation to control commands. Control commands specify desired collective thrust and body rates, the same control modality that the human pilots use. **a**, The perception system consists of a VIO module that computes a metric estimate of the drone state from camera images and high-frequency measurements obtained by an inertial measurement unit (IMU). The VIO estimate is coupled with a neural network that detects the corners of racing gates in the image stream. The corner detections are mapped to a 3D pose and fused with the VIO estimate using a Kalman filter. **b**, We use model-free on-policy deep RL to train the control policy in simulation. During training, the policy maximizes a reward that combines progress towards the centre of the next racing gate with a perception objective to keep the next gate in the field of view of the camera. To transfer the racing policy from simulation to the physical world, we augment the simulation with data-driven residual models of the vehicle's perception and dynamics. These residual models are identified from real-world experience collected on the race track. MLP, multilayer perceptron.

found perception residuals to be stochastic and dynamics residuals to be largely deterministic (Extended Data Fig. 1). These residual models are integrated into the simulation and the racing policy is fine-tuned in this augmented simulation. This approach is related to the empirical actuator models used for simulation-to-reality transfer in ref. 37 but further incorporates empirical modelling of the perception system and also accounts for the stochasticity in the estimate of the platform state.

We ablate each component of Swift in controlled experiments reported in the extended data. Also, we compare against recent work that tackles the task of autonomous drone racing with traditional methods, including trajectory planning and model predictive control (MPC). Although such approaches achieve comparable or even superior performance to our approach in idealized conditions, such as simplified dynamics and perfect knowledge of the robot's state, their performance collapses when their assumptions are violated. We find that approaches that rely on precomputed paths[28,29] are particularly sensitive to noisy perception and dynamics. No traditional method has achieved competitive lap times compared with Swift or human world champions, even when provided with highly accurate state estimation from a motion-capture system. Detailed analysis is provided in the extended data.

Results

The drone races take place on a track designed by an external world-class FPV pilot. The track features characteristic and challenging manoeuvres, such as a Split-S (Figs. 1a (top-right corner) and 4d). Pilots are allowed to continue racing even after a crash, provided their vehicle is still able to fly. If both drones crash and cannot complete the track, the drone that proceeded farther along the track wins.

As shown in Fig. 3b, Swift wins 5 out of 9 races against A. Vanover, 4 out of 7 races against T. Bitmatta and 6 out of 9 races against M. Schaepper. Out of the 10 losses recorded for Swift, 40% were because of a collision with the opponent, 40% because of collision with a gate and 20% because of the drone being slower than the human pilot. Overall, Swift

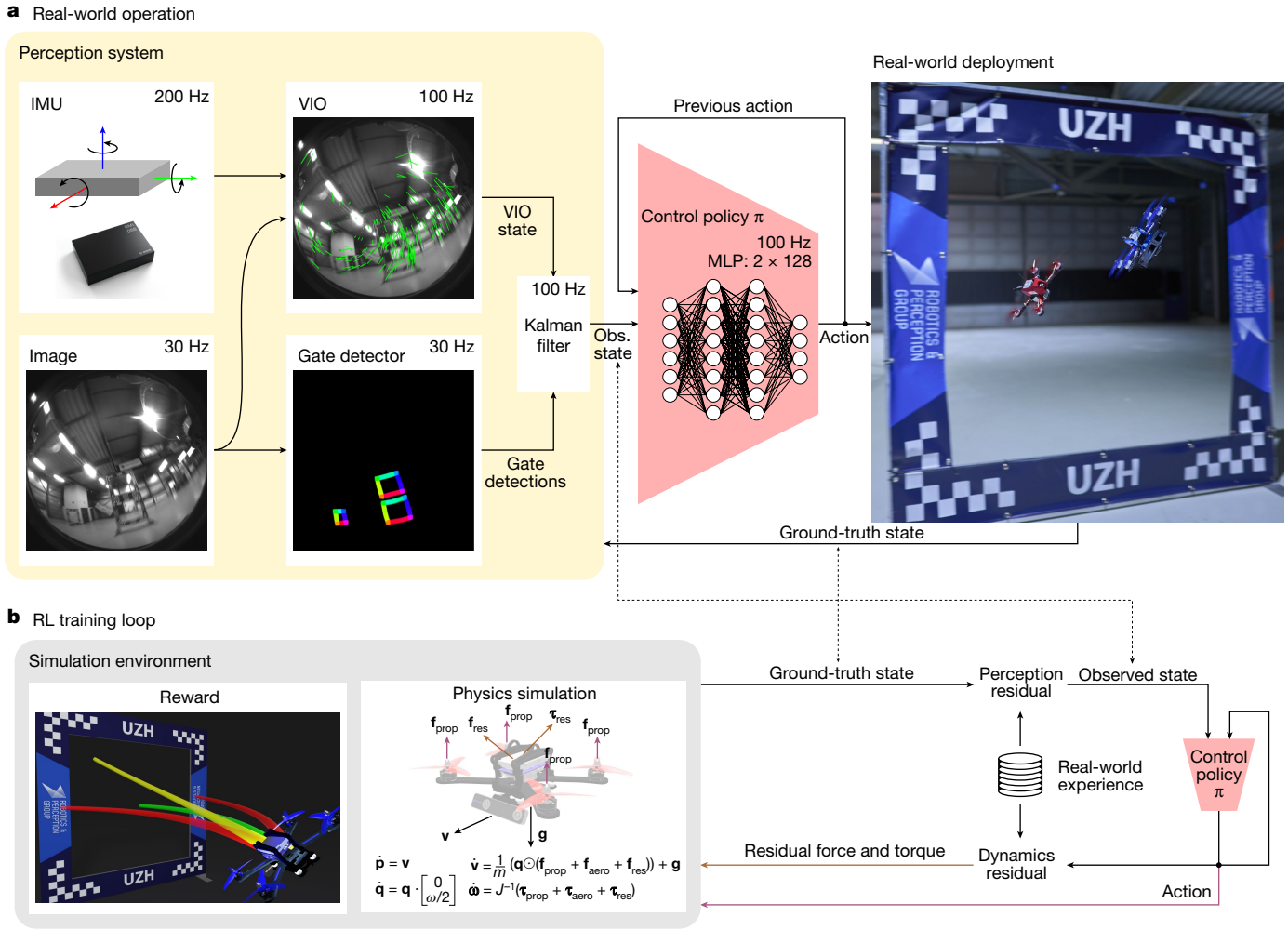


图 2 | Swift 系统。 Swift 由两个关键模块组成：将视觉和惯性信息转化为低维状态观察的感知系统，以及将状态观察映射到控制命令的控制策略。控制命令指定所需的集体推力和机身速率，这与人类飞行员使用的控制方式相同。 a，感知系统由一个 VIO 模块组成，该模块计算以下指标的度量估计

通过相机图像和惯性测量单元 (IMU) 获得的高频测量来确定无人机状态。 VIO 估计与检测图像中赛车门角点的神经网络相结合

溪流。角点检测被映射到 3D 位姿，并使用卡尔曼滤波器与 VIO 估计融合。 b，我们使用无模型的同策略深度强化学习来训练模拟中的控制策略。在训练过程中，该策略最大化奖励，将下一个赛车门中心的进展与将下一个赛车门保持在摄像机视野中的感知目标结合起来。为了将赛车策略从模拟转移到物理世界，我们使用数据驱动的车辆感知和动力学残差模型来增强模拟。这些残差模型是根据在赛道上收集的真实经验来识别的。 MLP，多层感知器。

发现感知残差是随机的，而动态残差在很大程度上是确定性的（扩展数据图 1）。这些残差模型被集成到模拟中，并且在该增强模拟中对赛车策略进行微调。这种方法与参考文献 37中用于模拟到现实转换的经验执行器模型相关，但进一步结合了感知系统的经验模型，并且还考虑了平台状态估计中的随机性。

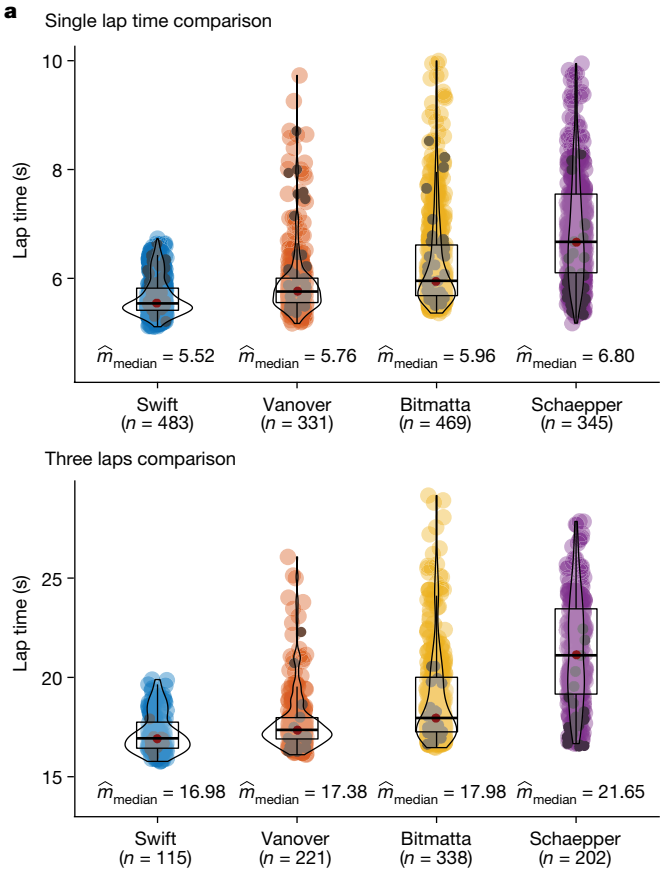
我们在扩展数据中报告的受控实验中消除了 Swift 的每个组件。此外，我们还与最近使用传统方法（包括轨迹规划和模型预测控制（MPC））解决自主无人机竞赛任务的工作进行了比较。尽管这些方法在理想条件下实现了与我们的方法相当甚至更好的性能，例如简化的动力学和对机器人状态的完美了解，但当它们的假设被违反时，它们的性能就会崩溃。我们发现依赖预先计算路径[28,29]的方法对噪声感知和动态特别敏感。传统方法没有

与 Swift 或人类世界冠军相比，即使提供了来自动作捕捉系统的高度准确的状态估计，也取得了有竞争力的单圈时间。扩展数据中提供了详细的分析。

结果

无人机比赛在由外部世界级 FPV 飞行员设计的赛道上进行。该赛道具有特色且具有挑战性的动作，例如 Split-S（图 1a（右上角）和 4d）。即使在事故发生后，只要他们的车辆仍然能够飞行，飞行员就可以继续比赛。如果两架无人机均坠毁且无法完成跑道，则沿跑道前进较远的无人机获胜。

如图 3b所示，Swift在对阵A. Vanover的9场比赛中赢了5场，在对阵T. Bitmatta的7场比赛中赢了4场，在对阵M. Schaepper的9场比赛中赢了6场。 在 Swift 记录的 10 次损失中，40% 是因为与对手相撞，40% 是因为与大门相撞，20% 是因为无人机比人类飞行员慢。总体而言，斯威夫特



b Head-to-head racing results

	Number of races	Best time-to-finish	Wins	Losses	Win ratio
A. Vanover versus Swift	9	17.956 s	4	5	0.44
T. Bitmatta versus Swift	7	18.746 s	3	4	0.43
M. Schaepper versus Swift	9	21.160 s	3	6	0.33
Swift versus human pilots	25	17.465 s	15	10	0.60

Fig. 3 | Results. a, Lap-time results. We compare Swift against the human pilots in time-trial races. Lap times indicate best single lap times and best average times achieved in a heat of three consecutive laps. The reported statistics are computed over a dataset recorded during one week on the race track, which corresponds to 483 (115) data points for Swift, 331 (221) for A. Vanover, 469 (338) for T. Bitmatta and 345 (202) for M. Schaepper. The first number is the number of single laps and the second is the number of three consecutive laps. The dark points in each distribution correspond to laps flown in race conditions. **b**, Head-to-head results. We report the number of head-to-head races flown by each pilot, the number of wins and losses, as well as the win ratio.

wins most races against each human pilot. Swift also achieves the fastest race time recorded, with a lead of half a second over the best time clocked by a human pilot (A. Vanover).

Figure 4 and Extended Data Table 1d provide an analysis of the fastest lap flown by Swift and each human pilot. Although Swift is globally faster than all human pilots, it is not faster on all individual segments of the track (Extended Data Table 1). Swift is consistently faster at the start and in tight turns such as the split S. At the start, Swift has a lower reaction time, taking off from the podium, on average, 120 ms before human pilots. Also, it accelerates faster and reaches higher speeds going into the first gate (Extended Data Table 1d, segment 1). In sharp turns, as shown in Fig. 4c,d, Swift finds tighter manoeuvres. One hypothesis is that Swift optimizes trajectories on a longer timescale than human pilots. It is known that model-free RL can optimize long-term rewards through a value function[38]. Conversely, human pilots plan their motion on a shorter timescale, up to one gate into the future[39]. This is apparent, for example in the split S (Fig. 4b,d), for which human pilots are faster in the beginning and at the end of the manoeuvre, but slower overall (Extended Data Table 1d, segment 3). Also, human pilots orient the aircraft to face the next gate earlier than Swift does (Fig. 4c,d). We propose that human pilots are accustomed to keeping the upcoming gate in view, whereas Swift has learned to execute some manoeuvres while relying on other cues, such as inertial data and visual odometry against features in the surrounding environments. Overall, averaged over the entire track, the autonomous drone achieves the highest average speed, finds the shortest racing line and manages to maintain the aircraft closer to its actuation limits throughout the race, as indicated by the average thrust and power drawn (Extended Data Table 1d).

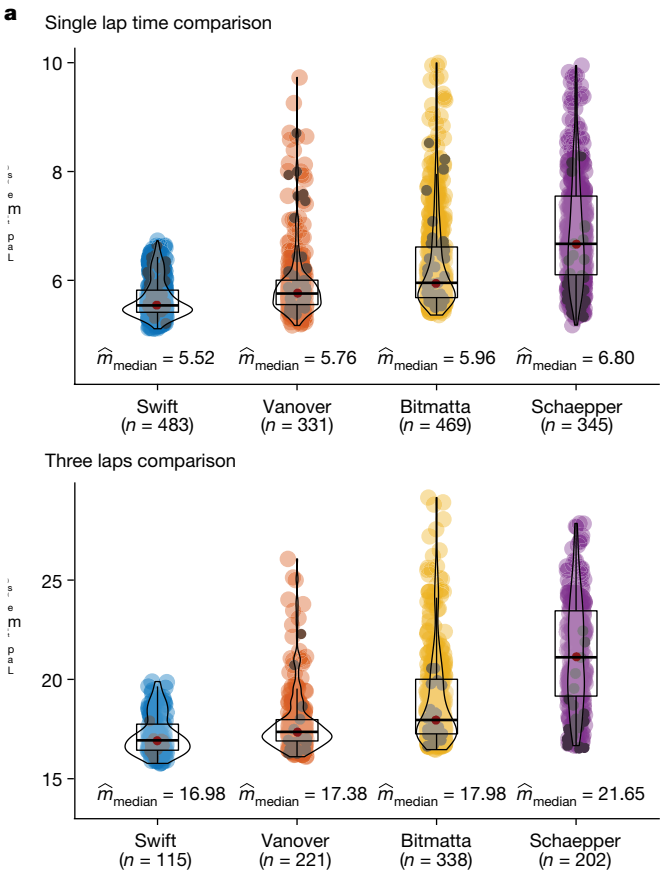
We also compare the performance of Swift and the human champions in time trials (Fig. 3a). In a time trial, a single pilot races the track, with the number of laps left to the discretion of the pilot. We accumulate time-trial data from the practice week and the races, including training runs (Fig. 3a, coloured) and laps flown in race conditions (Fig. 3a, black). For each contestant, we use more than 300 laps for computing statistics. The autonomous drone more consistently pushes for fast lap times, exhibiting lower mean and variance. Conversely, human pilots decide whether to push for speed on a lap-by-lap basis, yielding higher mean and variance in lap times, both during training and in the races. The ability to adapt the flight strategy allows human pilots to maintain a slower pace if they identify that they have a clear lead, so as to reduce the risk of a crash. The autonomous drone is unaware of its opponent and pushes for fastest expected completion time no matter what, potentially risking too much when in the lead and too little when trailing behind[40].

Discussion

FPV drone racing requires real-time decision-making based on noisy and incomplete sensory input from the physical environment. We have presented an autonomous physical system that achieves champion-level performance in this sport, reaching—and at times exceeding—the performance of human world champions. Our system has certain structural advantages over the human pilots. First, it makes use of inertial data from an onboard inertial measurement unit[32]. This is akin to the human vestibular system[41], which is not used by the human pilots because they are not physically in the aircraft and do not feel the accelerations acting on it. Second, our system benefits from lower sensorimotor latency (40 ms for Swift versus an average of 220 ms for expert human pilots[39]). On the other hand, the limited refresh rate of the camera used by Swift (30 Hz) can be considered a structural advantage for human pilots, whose cameras' refresh rate is four times as fast (120 Hz), improving their reaction time[42].

Human pilots are impressively robust: they can crash at full speed, and—if the hardware still functions—carry on flying and complete the track. Swift was not trained to recover after a crash. Human pilots are also robust to changes in environmental conditions, such as illumination, which can markedly alter the appearance of the track. By contrast, Swift's perception system assumes that the appearance of the environment is consistent with what was observed during training. If this assumption fails, the system can fail. Robustness to appearance changes can be provided by training the gate detector and the residual observation model in a diverse set of conditions. Addressing these limitations could enable applying the presented approach in autonomous drone racing competitions in which access to the environment and the drone is limited[25].

Notwithstanding the remaining limitations and the work ahead, the attainment by an autonomous mobile robot of world-champion-level performance in a popular physical sport is a milestone for robotics



乙 头对头比赛结果

	Number of races	Best time-to-finish	Wins	Losses	Win ratio
A. Vanover versus Swift	9	17.956 s	4	5	0.44
T. Bitmatta versus Swift	7	18.746 s	3	4	0.43
M. Schaepper versus Swift	9	21.160 s	3	6	0.33
Swift versus human pilots	25	17.465 s	15	10	0.60

图 3 | 结果。 a，单圈时间结果。我们在计时赛中将斯威夫特与人类飞行员进行了比较。单圈时间表示在连续三圈预赛中取得的最佳单圈时间和最佳平均时间。报告的统计数据是根据赛道上一周记录的数据集计算得出的，其中对应于 Swift 的 483 (115) 个数据点、A. Vanover 的 331 (221) 个数据点、T. Bitmatta 的 469 (338) 个数据点和 M. Schaepper 的 345 (202) 个数据点。第一个数字是单圈数，第二个数字是连续三圈数。 每个分布中的黑点对应于在比赛条件下飞行的圈数。

b，面对面的结果。我们报告每位飞行员参加的正面比赛次数、胜负次数以及胜率。

赢得与每位人类飞行员的大多数比赛。斯威夫特还创造了最快的比赛时间记录，比人类飞行员（A. Vanover）的最佳时间领先半秒。

图 4和扩展数据表 1d提供了对Swift和每位人类飞行员飞行的最快圈速的分析。尽管 Swift 在全球范围内比所有人类飞行员都快，但它并不是在赛道的所有单独路段上都更快（扩展数据表 1）。 Swift 在起步和急转弯（例如分体式 S）时始终更快。起步时，Swift 的反应时间较短，从讲台起飞，平均比人类飞行员早 120 ms。此外，它加速得更快，并在进入第一个门时达到更高的速度（扩展数据表 1d，段 1）。 在急转弯时，如图 4c,d所示，斯威夫特发现了更严格的机动。 一种假设是，Swift 在比人类飞行员更长的时间尺度上优化轨迹。众所周知，无模型强化学习可以优化通过价值函数[38]获得长期奖励。相反，人类飞行员在更短的时间尺度上规划他们的运动，最多可达未来的一扇门[39]。 这一点很明显，例如在分割 S 中（图 4b，d），人类飞行员在动作开始和结束时速度更快，但总体速度较慢（扩展数据表 1d，第 3 段）。此外，人类飞行员比 Swift 更早地将飞机定向到面向下一个登机口（图 4c，d）。我们认为人类飞行员习惯于将即将到来的大门保持在视野中，而斯威夫特已经学会了在依赖其他线索的同时执行一些机动，例如惯性数据和针对周围环境特征的视觉里程计。总体而言，在整个赛道上平均，自主无人机实现了最高的平均速度，找到最短的比赛线，并设法使飞机在整个比赛中保持接近其驱动极限，如平均推力和功率消耗所示（扩展数据表 1d）。

我们还比较了 Swift 和人类冠军在计时赛中的表现（图 3a）。在计时赛中，一名飞行员在赛道上比赛，圈数由飞行员自行决定。我们从练习周和比赛中积累计时赛数据，包括训练跑（图 3a，彩色）和在比赛条件下飞行的圈数（图 3a，黑色）。对于每位参赛者，我们使用超过300圈进行计算统计。自主无人机更一致地推动更快的单圈时间，表现出更低的均值和方差。相反，人类飞行员决定是否逐圈提高速度，从而在训练和比赛中产生更高的单圈时间平均值和方差。调整飞行策略的能力使人类飞行员在确定自己明显领先时可以保持较慢的速度，从而降低坠机风险。自主无人机不知道其对手，无论如何都会争取最快的预期完成时间，领先时可能会冒太大的风险，而落后时则可能会冒太小的风险[40]。

讨论

FPV 无人机竞赛需要基于来自物理环境的嘈杂和不完整的感官输入进行实时决策。我们提出了一种自主身体系统，可以在这项运动中实现冠军级别的表现，达到甚至有时超过人类世界冠军的表现。我们的系统比人类飞行员具有一定的结构优势。首先，它利用来自机载惯性测量单元[32]的惯性数据。这类似于人类前庭系统[41]，人类飞行员不使用该系统，因为他们实际上并不在飞机上，也感觉不到作用在飞机上的加速度。其次，我们的系统受益于较低的感觉运动延迟（Swift 为 40 ms，而人类专家飞行员的平均延迟为 220 ms[39]）。 另一方面，Swift 使用的摄像头有限的刷新率（30 Hz）对于人类飞行员来说可以被视为结构性优势，其摄像头的刷新率是其四倍（120 Hz），从而缩短了他们的反应时间[42]。

人类飞行员的鲁棒性令人印象深刻：他们可以全速坠毁，并且——如果硬件仍然正常工作——继续飞行并完成跑道。斯威夫特没有接受过撞车后恢复的训练。人类飞行员对环境条件的变化也很敏感，例如照明，这可以显着改变轨道的外观。相比之下，Swift 的感知系统假设环境的外观与训练期间观察到的一致。如果这个假设失败，系统就会失败。通过在不同的条件下训练门检测器和残差观察模型可以提供对外观变化的鲁棒性。解决这些限制可以将所提出的方法应用于自主无人机竞赛，在这些竞赛中，对环境和无人机的访问受到限制[25]。

尽管还存在一些限制和未来的工作，自主移动机器人在一项流行的体育运动中取得世界冠军级的表现对于机器人技术来说是一个里程碑

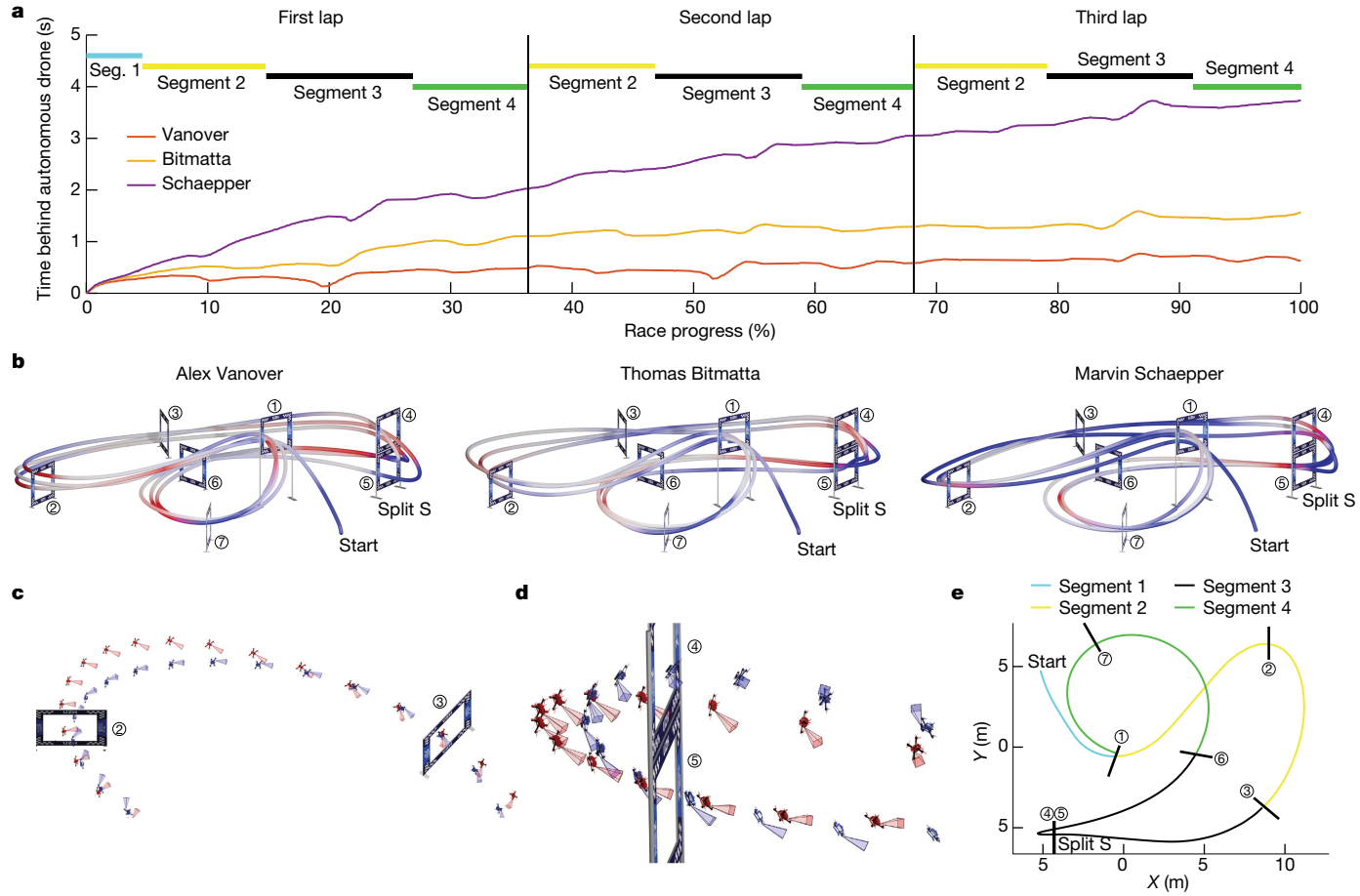


Fig. 4 | Analysis. a, Comparison of the fastest race of each pilot, illustrated by the time behind Swift. The time difference from the autonomous drone is computed as the time since it passed the same position on the track. Although Swift is globally faster than all human pilots, it is not necessarily faster on all individual segments of the track. **b**, Visualization of where the human pilots are faster (red) and slower (blue) compared with the autonomous drone. Swift is consistently faster at the start and in tight turns, such as the split S. **c**, Analysis of the manoeuvre after gate 2. Swift in blue, Vanover in red. Swift gains time against human pilots in this segment as it executes a tighter turn while maintaining comparable speed. **d**, Analysis of the split S manoeuvre. Swift in blue, Vanover in red. The split S is the most challenging segment in the race track, requiring a carefully coordinated roll and pitch motion that yields a descending half-loop through the two gates. Swift gains time against human pilots on this segment as it executes a tighter turn with less overshoot. **e**, Illustration of track segments used for analysis. Segment 1 is traversed once at the start, whereas segments 2–4 are traversed in each lap (three times over the course of a race).

and machine intelligence. This work may inspire the deployment of hybrid learning-based solutions in other physical systems, such as autonomous ground vehicles, aircraft and personal robots, across a broad range of applications.

Online content

Any methods, additional references, Nature Portfolio reporting summaries, source data, extended data, supplementary information, acknowledgements, peer review information; details of author contributions and competing interests; and statements of data and code availability are available at https://doi.org/10.1038/s41586-023-06419-4.

1. De Wagter, C., Paredes-Vallés, F., Sheth, N. & de Croon, G. Learning fast in autonomous drone racing. *Nat. Mach. Intell.* **3**, 923 (2021).
2. Hanover, D. et al. Autonomous drone racing: a survey. Preprint at https://arxiv.org/abs/2301.01755 (2023).
3. Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (MIT Press, 2018).
4. Mnih, V. et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).
5. Schrittwieser, J. et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature* **588**, 604–609 (2020).
6. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O. & Clune, J. First return, then explore. *Nature* **590**, 580–586 (2021).
7. Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).
8. Silver, D. et al. Mastering the game of Go without human knowledge. *Nature* **550**, 354–359 (2017).
9. Silver, D. et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **362**, 1140–1144 (2018).
10. Vinyals, O. et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **575**, 350–354 (2019).
11. Berner, C. et al. Dota 2 with large scale deep reinforcement learning. Preprint at https://arxiv.org/abs/1912.06680 (2019).
12. Fuchs, F., Song, Y., Kaufmann, E., Scaramuzza, D. & Dürr, P. Super-human performance in Gran Turismo Sport using deep reinforcement learning. *IEEE Robot. Autom. Lett.* **6**, 4257–4264 (2021).
13. Wurman, P. R. et al. Outracing champion Gran Turismo drivers with deep reinforcement learning. *Nature* **602**, 223–228 (2022).
14. Funke, J. et al. in *Proc. 2012 IEEE Intelligent Vehicles Symposium* 541–547 (IEEE, 2012).
15. Spielberg, N. A., Brown, M., Kapania, N. R., Kegelman, J. C. & Gerdes, J. C. Neural network vehicle models for high-performance automated driving. *Sci. Robot.* **4**, eaaw1975 (2019).
16. Won, D.-O., Müller, K.-R. & Lee, S.-W. An adaptive deep reinforcement learning framework enables curling robots with human-like performance in real-world conditions. *Sci. Robot.* **5**, eabb9764 (2020).
17. Moon, H., Sun, Y., Baltes, J. & Kim, S. J. The IROS 2016 competitions. *IEEE Robot. Autom. Mag.* **24**, 20–29 (2017).
18. Jung, S., Hwang, S., Shin, H. & Shim, D. H. Perception, guidance, and navigation for indoor autonomous drone racing using deep learning. *IEEE Robot. Autom. Lett.* **3**, 2539–2544 (2018).
19. Kaufmann, E. et al. in *Proc. 2nd Conference on Robot Learning (CoRL)* 133–145 (PMLR, 2018).
20. Zhang, D. & Doyle, D. D. in *Proc. 2020 IEEE Aerospace Conference*, 1–11 (IEEE, 2020).

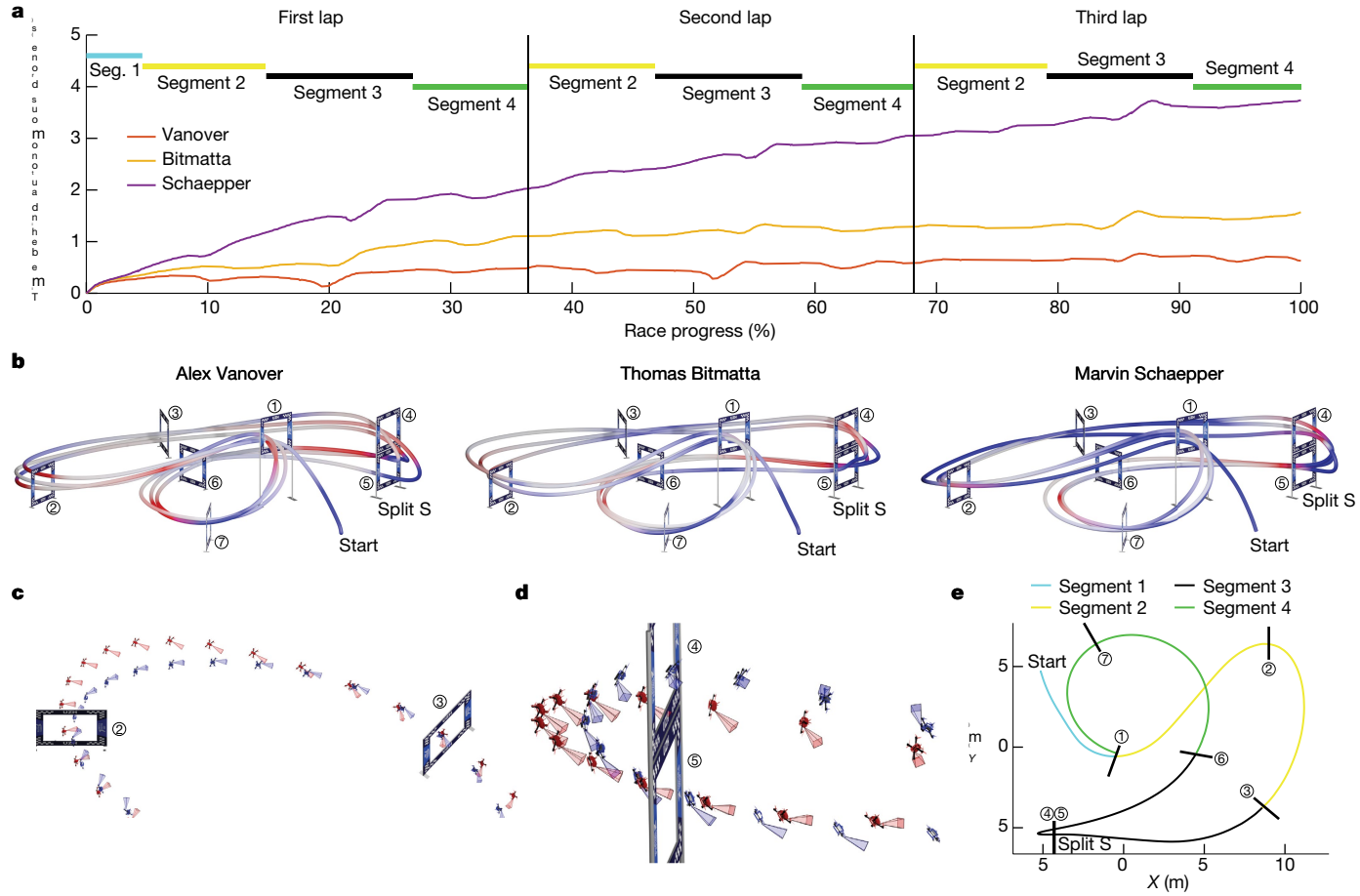


图 4 | 分析。 a，每位飞行员最快比赛的比较，以落后于斯威夫特的时间来说明。与自主无人机的时间差计算为自其通过轨道上相同位置以来的时间。尽管斯威夫特在全球范围内比所有人类飞行员都快，但它不一定在赛道的所有单独路段上都更快。 b，人类飞行员所在位置的可视化

与自主无人机相比更快（红色）和更慢（蓝色）。斯威夫特在起步和急转弯时始终更快，例如分叉 S.c，2 号门后的机动分析。斯威夫特为蓝色，范诺弗为红色。斯威夫特在这方面比人类飞行员赢得了时间，因为它执行了更急的转弯，而保持可比较的速度。 d，分体S动作分析。斯威夫特为蓝色，凡诺弗为红色。 S 型赛道是赛道中最具挑战性的路段，需要仔细协调滚动和俯仰运动，从而通过两个门产生下降的半环。斯威夫特在这一段上比人类飞行员赢得了时间，因为它执行了更紧的转弯，超调量更少。 e，用于分析的轨道段的图示。赛段 1 在开始时经过一次，而赛段 2-4 每圈经过一次（在比赛过程中经过 3 次）。

和机器智能。这项工作可能会激发在其他物理系统中部署基于混合学习的解决方案，例如自主地面车辆、飞机和个人机器人等广泛的应用。

在线内容

任何方法、附加参考文献、自然组合报告摘要、源数据、扩展数据、补充信息、致谢、同行评审信息；作者贡献和竞争利益的详细信息；数据和代码可用性声明可在 https://doi.org/10.1038/s41586-023-06419-4 上获取。

1. De Wagter, C.、Paredes-Vallés, F.、Sheth, N. 和 de Croon, G. 在自主无人机竞赛中快速学习。 *Nat. Mach. Intell.* 3, 923 (2021)。 2. Hanover, D. 等人。自主无人机竞赛：一项调查。预印本位于 https://arxiv.org/abs/2301.01755 (2023)。 3. Sutton, R. S. 和 Barto, A. G. *Reinforcement Learning: An Introduction*（麻省理工学院出版社，2018 年）。 4. Mnih, V. 等人。通过深度强化学习实现人类水平的控制。 *Nature* 518, 529–533 (2015)。 5. Schrittwieser, J. 等人。通过学习模型进行规划，掌握 Atari、围棋、国际象棋和将棋。 *Nature* 588, 604–609 (2020)。 6. Ecoffet, A.、Huizinga, J.、Lehman, J.、Stanley, K. O. 和 Clune, J. 首先返回，然后探索。 *Nature* 590, 580–586 (2021)。

7. Silver, D. 等人。通过深度神经网络和树搜索掌握围棋游戏。 *Nature* 529, 484–489 (2016)。 8. Silver, D. 等人。在没有人类知识的情况下掌握围棋游戏。 *Nature* 550, 354–359 (2017)。 9. Silver, D. 等人。一种通用强化学习算法，通过自对弈掌握国际象棋、将棋和围棋。 *Science* 362, 1140–1144 (2018)。 10. Vinyals, O. 等人。使用多智能体强化学习的星际争霸 II 大师级别。 *Nature* 575, 350–354 (2019)。 11. Berner, C. 等人。 Dota 2 具有大规模深度强化学习。预印本位于 https://arxiv.org/abs/1912.06680 (2019)。 12. Fuchs, F.、Song, Y.、Kaufmann, E.、Scaramuzza, D. 和 Dürr, P。使用深度强化学习在 Gran Turismo Sport 中实现超人表现。 *IEEE Robot. Autom. Lett.* 6, 4257–4264 (2021)。 13. Wurman, P. R. 等人。通过深度强化学习超越 Gran Turismo 冠军车手。 *Nature* 602, 223–228 (2022)。 14. Funke, J. 等人。在 *Proc. 2012 IEEE Intelligent Vehicles Symposium* 541–547（IEEE，2012）中。 15. Spielberg, N. A.、Brown, M.、Kapania, N. R.、Kegelman, J. C. 和 Gerdes, J. C. 用于高性能自动驾驶的神经网络车辆模型。 *Sci. Robot.* 4，eaaw1975 (2019)。 16. Won，D.-O.，Müller，K.-R。 &李，S.-W。自适应深度强化学习框架使冰壶机器人在现实条件下具有类似人类的表现。 *Sci. Robot.* 5，eabb9764 (2020)。 17. Moon, H.、Sun, Y.、Baltes, J. 和 Kim, S. J. IROS 2016 比赛。 *IEEE Robot. Autom. Mag.* 24, 20–29 (2017)。 18. Jung, S., Hwang, S., Shin, H. & Shim, D. H. 室内感知、引导和导航使用深度学习的自主无人机竞赛。 *IEEE Robot. Autom. Lett.* 3, 2539–2544 (2018)。 19. Kaufmann, E. 等人。见 *Proc. 2nd Conference on Robot Learning (CoRL)* 133–145（PMLR，2018）。 20. 张 D. 和 Doyle, D. D.，《*Proc. 2020 IEEE Aerospace Conference*》，1–11（IEEE，2020）。

21. Loquercio, A. et al. Deep drone racing: from simulation to reality with domain randomization. *IEEE Trans. Robot.* **36**, 1–14 (2019).
22. Loquercio, A. et al. Learning high-speed flight in the wild. *Sci. Robot.* **6**, eabg5810 (2021).
23. Kaufmann, E. et al. in *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 690–696 (IEEE, 2019).
24. Li, S., van der Horst, E., Duernay, P., De Wagter, C. & de Croon, G. C. Visual model-predictive localization for computationally efficient autonomous racing of a 72-g drone. *J. Field Robot.* **37**, 667–692 (2020).
25. A.I. is flying drones (very, very slowly). https://www.nytimes.com/2019/03/26/technology/alphapilot-ai-drone-racing.html (2019).
26. Foehn, P. et al. AlphaPilot: autonomous drone racing. *Auton. Robots* **46**, 307–320 (2021).
27. Wagter, C. D., Paredes-Vallé, F., Sheth, N. & de Croon, G. The sensing, state-estimation, and control behind the winning entry to the 2019 Artificial Intelligence Robotic Racing Competition. *Field Robot.* **2**, 1263–1290 (2022).
28. Foehn, P., Romero, A. & Scaramuzza, D. Time-optimal planning for quadrotor waypoint flight. *Sci. Robot.* **6**, eabh1221 (2021).
29. Romero, A., Sun, S., Foehn, P. & Scaramuzza, D. Model predictive contouring control for time-optimal quadrotor flight. *IEEE Trans. Robot.* **38**, 3340–3356 (2022).
30. Sun, S., Romero, A., Foehn, P., Kaufmann, E. & Scaramuzza, D. A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. *IEEE Trans. Robot.* **38**, 3357–3373 (2021).
31. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. Preprint at https://arxiv.org/abs/1707.06347 (2017).
32. Scaramuzza, D. & Zhang, Z. *Encyclopedia of Robotics* (eds Ang, M., Khatib, O. & Siciliano, B.) 1–9 (Springer, 2019).
33. Huang, G. in *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 9572–9582 (IEEE, 2019).
34. Collins, T. & Bartoli, A. Infinitesimal plane-based pose estimation. *Int. J. Comput. Vis.* **109**, 252–286 (2014).
35. Song, Y., Steinweg, M., Kaufmann, E. & Scaramuzza, D. in *Proc. 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 1205–1212 (IEEE, 2021).
36. Williams, C. K. & Rasmussen, C. E. *Gaussian Processes for Machine Learning* (MIT Press, 2006).
37. Hwangbo, J. et al. Learning agile and dynamic motor skills for legged robots. *Sci. Robot.* **4**, eaau5872 (2019).
38. Hung, C.-C. et al. Optimizing agent behavior over long time scales by transporting value. *Nat. Commun.* **10**, 5223 (2019).
39. Pfeiffer, C. & Scaramuzza, D. Human-piloted drone racing: visual processing and control. *IEEE Robot. Autom. Lett.* **6**, 3467–3474 (2021).
40. Spica, R., Cristofalo, E., Wang, Z., Montijano, E. & Schwager, M. A real-time game theoretic planner for autonomous two-player drone racing. *IEEE Trans. Robot.* **36**, 1389–1403 (2020).
41. Day, B. L. & Fitzpatrick, R. C. The vestibular system. *Curr. Biol.* **15**, R583–R586 (2005).
42. Kim, J. et al. Esports arms race: latency and refresh rate for competitive gaming tasks. *J. Vis.* **19**, 218c (2019).

21. Loquercio, A. 等人。深度无人机竞赛：通过域随机化从模拟到现实。 *IEEE Trans. Robot.* 36, 1–14 (2019)。 22. Loquercio, A. 等人。在野外学习高速飞行。 *Sci. Robot.* 6，eabg5810 (2021)。 23. Kaufmann, E. 等人。在 *Proc. 2019 International Conference on Robotics and Automation (ICRA)* 690–696（IEEE，2019）中。 24. Li, S.、van der Horst, E.、Duernay, P.、De Wagter, C. 和 de Croon, G. C. 用于计算高效的 72 克无人机自主竞赛的视觉模型预测定位。 *J. Field Robot.* 37, 667–692 (2020)。 25. 人工智能正在驾驶无人机（非常非常慢）。 https://www.nytimes.com/2019/03/26/technology/alphapilot-ai-drone-racing.html（2019）。 26.Foehn, P. 等人。 AlphaPilot：自主无人机竞赛。 *Auton. Robots* 46, 307–320 (2021)。 27. Wagter, C. D.、Paredes-Vallé, F.、Sheth, N. 和 de Croon, G. 2019 年人工智能机器人赛车竞赛获奖作品背后的传感、状态估计和控制。 *Field Robot.* 2, 1263–1290 (2022)。 28. Foehn, P.、Romero, A. 和 Scaramuzza, D. 四旋翼飞行器航路点飞行的时间最优规划。 *Sci. Robot.* 6，eabh1221 (2021)。 29. Romero, A.、Sun, S.、Foehn, P. 和 Scaramuzza, D. 时间最佳四旋翼飞行器飞行的模型预测轮廓控制。 *IEEE Trans. Robot.* 38, 3340–3356 (2022)。 30. Sun, S.、Romero, A.、Foehn, P.、Kaufmann, E. 和 Scaramuzza, D. 四旋翼敏捷飞行的非线性 MPC 和基于微分平坦度的控制的比较研究。 *IEEE Trans. Robot.* 38, 3357–3373 (2021)。 31. Schulman, J.、Wolski, F.、Dhariwal, P.、Radford, A. 和 Klimov, O. 近端策略优化算法。预印本https://arxiv.org/abs/1707.06347 (2017)。 32. Scaramuzza, D. 和Zhang, Z. *Encyclopedia of Robotics*（Ang, M.、Khatib, O. 和 Siciliano, B. 编辑）1–9（Springer，2019 年）。 33. Huang, G.,
Proc. 2019 International Conference on Robotics and Automation (ICRA) 9572–9582（IEEE，2019）。 34. Collins, T. 和 Bartoli, A. 基于无穷小平面的姿态估计。 *Int. J. Comput. Vis.* 109, 252–286 (2014)。 35. Song, Y.、Steinweg, M.、Kaufmann, E. 和 Scaramuzza, D.，见
Proc. 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) 1205–1212（IEEE，2021）。
36. Williams, C. K. 和 Rasmussen, C. E. *Gaussian Processes for Machine Learning*（麻省理工学院出版社，2006 年）。 37. Hwangbo, J. 等人。学习腿式机器人的敏捷和动态运动技能。 *Sci. Robot.* 4，eaau5872（2019）。 38. 洪，C.-C。等 等。通过传输价值来优化代理的长期行为。 *Nat. Commun.* 10, 5223 (2019)。 39. Pfeiffer, C. 和 Scaramuzza, D. 人类驾驶的无人机竞赛：视觉处理和控制。 *IEEE Robot. Autom. Lett.* 6, 3467–3474 (2021)。 40. Spica, R., Cristofalo, E., Wang, Z., Montijano, E. & Schwager, M. 实时博弈论

自主两人无人机竞赛的规划器。 *IEEE Trans. Robot.* 36, 1389–1403 (2020)。 41. Day, B. L. 和 Fitzpatrick, R. C. 前庭系统。 *Curr. Biol.* 15，R583–R586 (2005)。 42. Kim, J. 等人。电子竞技军备竞赛：竞技游戏任务的延迟和刷新率。 *J. Vis.* 19, 218c (2019)。

Methods

Quadrotor simulation

Quadrotor dynamics. To enable large-scale training, we use a high-fidelity simulation of the quadrotor dynamics. This section briefly explains the simulation. The dynamics of the vehicle can be written as

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \quad (1)$$

in which $\odot$ represents quaternion rotation, $\mathbf{p}_{\mathcal{WB}}$, $\mathbf{q}_{\mathcal{WB}}$, $\mathbf{v}_{\mathcal{W}}$ and $\boldsymbol{\omega}_{\mathcal{B}}$ denote the position, attitude quaternion, inertial velocity and body rates of the quadcopter, respectively. The motor time constant is k_{mot} and the motor speeds $\boldsymbol{\Omega}$ and $\boldsymbol{\Omega}_{\text{ss}}$ are the actual and steady-state motor speeds, respectively. The matrix J is the inertia of the quadcopter and $\mathbf{g}_{\mathcal{W}}$ denotes the gravity vector. Two forces act on the quadrotor: the lift force $\mathbf{f}_{\text{prop}}$ generated by the propellers and an aerodynamic force $\mathbf{f}_{\text{aero}}$ that aggregates all other forces, such as aerodynamic drag, dynamic lift and induced drag. The torque is modelled as a sum of four components: the torque generated by the individual propeller thrusts $\boldsymbol{\tau}_{\text{prop}}$, the yaw torque $\boldsymbol{\tau}_{\text{mot}}$ generated by a change in motor speed, an aerodynamic torque $\boldsymbol{\tau}_{\text{aero}}$ that accounts for various aerodynamic effects such as blade flapping and an inertial term $\boldsymbol{\tau}_{\text{iner}}$. The individual components are given as

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i, \ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \quad (2)$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i, \ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}_{\mathcal{B}} \quad (3)$$

in which $\mathbf{r}_{\text{P},i}$ is the location of propeller i, expressed in the body frame, and $\mathbf{f}_i$ and $\boldsymbol{\tau}_i$ are the forces and torques, respectively, generated by the ith propeller. The axis of rotation of the ith motor is denoted by $\boldsymbol{\zeta}_i$, the combined inertia of the motor and propeller is $J_{\text{m+p}}$ and the derivative of the ith motor speed is $\dot{\Omega}_i$. The individual propellers are modelled using a commonly used quadratic model, which assumes that the lift force and drag torque are proportional to the square of the propeller speed Ω_i:

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^{\top}, \quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^{\top} \quad (4)$$

in which c_{l} and c_{d} denote the propeller lift and drag coefficients, respectively.

Aerodynamic forces and torques. The aerodynamic forces and torques are difficult to model with a first-principles approach. We thus use a data-driven model[43]. To maintain the low computational complexity required for large-scale RL training, a grey-box polynomial model is used rather than a neural network. The aerodynamic effects are assumed to primarily depend on the velocity $\mathbf{v}_{\mathcal{B}}$ (in the body frame) and the average squared motor speed $\overline{\Omega^2}$. The aerodynamic forces f_x, f_y and f_z and torques τ_x, τ_y and τ_z are estimated in the body frame. The quantities v_x, v_y and v_z denote the three axial velocity components (in the body frame) and v_{xy} denotes the speed in the (x, y) plane of the quadrotor. On the basis of insights from the underlying physical processes, linear and quadratic combinations of the individual terms are selected. For readability, the coefficients multiplying each summand have been omitted:

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\,\overline{\Omega^2} + v_z\,\overline{\Omega^2} + v_{xy}\,v_z\,\overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} + v_y|v_y|\,\overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} + v_x|v_x|\,\overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

The respective coefficients are then identified from real-world flight data, in which motion capture is used to provide ground-truth forces and torque measurements. We use data from the race track, allowing the dynamics model to fit the track. This is akin to the human pilots' training for days or weeks before the race on the specific track that they will be racing. In our case, the human pilots are given a week of practice on the same track ahead of the competition.

Betaflight low-level controller. To control the quadrotor, the neural network outputs collective thrust and body rates. This control signal is known to combine high agility with good robustness to simulation-to-reality transfer[44]. The predicted collective thrust and body rates are then processed by an onboard low-level controller that computes individual motor commands, which are subsequently translated into analogue voltage signals through an electronic speed controller (ESC) that controls the motors. On the physical vehicle, this low-level proportional–integral–derivative (PID) controller and ESC are implemented using the open-source Betaflight and BLHeli32 firmware[45]. In simulation, we use an accurate model of both the low-level controller and the motor speed controller.

Because the Betaflight PID controller has been optimized for human-piloted flight, it exhibits some peculiarities, which the simulation correctly captures: the reference for the D-term is constantly zero (pure damping), the I-term gets reset when the throttle is cut and, under motor thrust saturation, the body rate control is assigned priority (proportional downscaling of all motor signals to avoid saturation). The gains of the controller used for simulation have been identified from the detailed logs of the Betaflight controller's internal states. The simulation can predict the individual motor commands with less than 1% error.

Battery model and ESC. The low-level controller converts the individual motor commands into a pulse-width modulation (PWM) signal and sends it to the ESC, which controls the motors. Because the ESC does not perform closed-loop control of the motor speeds, the steady-state motor speed $\Omega_{i,\text{ss}}$ for a given PWM motor command cmd_i is a function of the battery voltage. Our simulation thus models the battery voltage using a grey-box battery model[46] that simulates the voltage based on instantaneous power consumption P_{mot}:

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \quad (5)$$

The battery model[46] then simulates the battery voltage based on this power demand. Given the battery voltage U_{bat} and the individual motor command $u_{\text{cmd},i}$, we use the mapping (again omitting the coefficients multiplying each summand)

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \quad (6)$$

to calculate the corresponding steady-state motor speed $\Omega_{i,\text{ss}}$ required for the dynamics simulation in equation (1). The coefficients have been

方法

四旋翼飞行器模拟

四旋翼飞行器动力学。为了实现大规模训练，我们使用四旋翼飞行器动力学的高保真模拟。本节简要说明模拟。车辆的动力学可以写为

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \quad (1)$$

其中，$\odot$ 表示四元数旋转，$\mathbf{p}_{\mathcal{WB}}$、$\mathbf{q}_{\mathcal{WB}}$、$\mathbf{v}_{\mathcal{W}}$、$\boldsymbol{\omega}_{\mathcal{B}}$ 分别表示四轴飞行器的位置、姿态四元数、惯性速度和机身速率。电机时间常数为k_{mot}，电机速度$\boldsymbol{\Omega}$和$\boldsymbol{\Omega}_{\text{ss}}$分别是实际电机速度和稳态电机速度。矩阵$J$是四轴飞行器的惯性，$\mathbf{g}_{\mathcal{W}}$表示重力矢量。有两个力作用在四旋翼飞行器上：螺旋桨产生的升力 $\mathbf{f}_{\text{prop}}$ 和聚集所有其他力（例如气动阻力、动态升力和诱导阻力）的空气动力 $\mathbf{f}_{\text{aero}}$。扭矩被建模为四个分量的总和：各个螺旋桨推力产生的扭矩$\boldsymbol{\tau}_{\text{prop}}$、电机速度变化产生的偏航扭矩$\boldsymbol{\tau}_{\text{mot}}$、考虑各种空气动力效应（例如桨叶扑动）的空气动力扭矩$\boldsymbol{\tau}_{\text{aero}}$和惯性项$\boldsymbol{\tau}_{\text{iner}}$。各个组件给出为

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i, \ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{P,i} \times \mathbf{f}_i, \quad (2)$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i, \ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}_{\mathcal{B}} \quad (3)$$

其中$\mathbf{r}_{P,i}$是螺旋桨i的位置，以机身坐标系表示，$\mathbf{f}_i$和$\boldsymbol{\tau}_i$分别是第i螺旋桨产生的力和扭矩。第 i 个电机的旋转轴用 $\boldsymbol{\zeta}_i$ 表示，电机和螺旋桨的组合惯量为$J_{\text{m+p}}$，第 i 个电机速度的导数为 $\dot{\Omega}_i$。各个螺旋桨使用常用的二次模型进行建模，该模型假设升力和阻力扭矩与螺旋桨速度 Ω_i 的平方成正比：

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^\top, \quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^\top \quad (4)$$

其中c_{l}和c_{d}分别表示螺旋桨升力和阻力系数。

空气动力和扭矩。空气动力和扭矩很难用第一原理方法进行建模。因此，我们使用数据驱动模型[43]。为了保持大规模 RL 训练所需的低计算复杂度，使用灰盒多项式模型而不是神经网络。假设空气动力学效应主要取决于速度 $\mathbf{v}_{\mathcal{B}}$（在车身框架中）和平均平方电机速度 Ω^2。气动力f_x、f_y和f_z以及扭矩τ_x、τ_y和τ_z是在车身框架中估计的。量 v_x、v_y 和 v_z 表示三个轴向速度分量（在机身坐标系中），v_{xy} 表示四旋翼飞行器 (x、y) 平面中的速度。根据对底层物理过程的了解，选择各个项的线性和二次组合。为了便于阅读，省略了与每个被加数相乘的系数：

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\,\overline{\Omega^2} + v_z\,\overline{\Omega^2} + v_{xy}\,v_z\,\overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\,\overline{\Omega^2} + v_y|v_y|\,\overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\,\overline{\Omega^2} + v_x|v_x|\,\overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

然后从真实世界的飞行数据中识别相应的系数，其中运动捕捉用于提供地面真实力和扭矩测量。 我们使用来自赛道的数据，使动力学模型能够适应赛道。这类似于人类飞行员在比赛前在他们将要比赛的特定赛道上进行数天或数周的训练。在我们的例子中，人类飞行员在比赛前在同一跑道上进行了一周的练习。

Betaflight 低级控制器。为了控制四旋翼飞行器，神经网络输出集体推力和机身速率。 众所周知，该控制信号将高敏捷性与良好的鲁棒性结合起来，以实现从模拟到现实的转换[44]。然后，预测的集体推力和车身速率由机载低级控制器处理，该控制器计算各个电机命令，随后通过控制电机的电子速度控制器（ESC）将其转换为模拟电压信号。在实体车辆上，这种低级比例积分微分 (PID) 控制器和 ESC 是使用开源 Betaflight 和 BLHeli32 固件[45] 实现的。在仿真中，我们使用低级控制器和电机速度控制器的精确模型。

由于 Betaflight PID 控制器已针对人类驾驶飞行进行了优化，因此它表现出一些特性，仿真正确捕获了这些特性：D 项的参考始终为零（纯阻尼），I 项在油门关闭时重置，并且在电机推力饱和时，机身速率控制被分配优先级（所有电机信号按比例缩小以避免饱和）。 用于仿真的控制器的增益已从 Betaflight 控制器内部状态的详细日志中确定。 仿真可以预测各个电机命令，误差小于 1%。

电池型号和电调。低级控制器将各个电机命令转换为脉宽调制 (PWM) 信号，并将其发送到控制电机的 ESC。由于 ESC 不对电机速度执行闭环控制，因此给定 PWM 电机命令 cmd_i 的稳态电机速度 $\Omega_{i,\text{ss}}$ 是电池电压的函数。因此，我们的模拟使用灰盒电池模型 [46] 对电池电压进行建模，该模型根据瞬时功耗 P_{mot} 模拟电压：

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \quad (5)$$

电池模型[46]然后根据此模拟电池电压
电力需求。给定电池电压 U_{bat} 和单个电机命令 $u_{\text{cmd},i}$，我们使用映射（再次省略系数将每个被加数相乘）

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \quad (6)$$

计算所需的相应稳态电机速度$\boldsymbol{\Omega}_{i,\text{ss}}$
用于方程(1)中的动力学模拟。系数已

identified from Betaflight logs containing measurements of all involved quantities. Together with the model of the low-level controller, this enables the simulator to correctly translate an action in the form of collective thrust and body rates to desired motor speeds Ω_{ss} in equation (1).

Policy training

We train deep neural control policies that directly map observations $\mathbf{o}_t$ in the form of platform state and next gate observation to control actions $\mathbf{u}_t$ in the form of mass-normalized collective thrust and body rates[44]. The control policies are trained using model-free RL in simulation.

Training algorithm. Training is performed using proximal policy optimization[31]. This actor-critic approach requires jointly optimizing two neural networks during training: the policy network, which maps observations to actions, and the value network, which serves as the 'critic' and evaluates actions taken by the policy. After training, only the policy network is deployed on the robot.

Observations, actions and rewards. An observation $\mathbf{o}_t \in \mathbb{R}^{31}$ obtained from the environment at time t consists of: (1) an estimate of the current robot state; (2) the relative pose of the next gate to be passed on the track layout; and (3) the action applied in the previous step. Specifically, the estimate of the robot state contains the position of the platform, its velocity and attitude represented by a rotation matrix, resulting in a vector in $\mathbb{R}^{15}$. Although the simulation uses quaternions internally, we use a rotation matrix to represent attitude to avoid ambiguities[47]. The relative pose of the next gate is encoded by providing the relative position of the four gate corners with respect to the vehicle, resulting in a vector in $\mathbb{R}^{12}$. All observations are normalized before being passed to the network. Because the value network is only used during training time, it can access privileged information about the environment that is not accessible to the policy[48]. This privileged information is concatenated with other inputs to the policy network and contains the exact position, orientation and velocity of the robot.

For each observation $\mathbf{o}_t$, the policy network produces an action $\mathbf{a}_t \in \mathbb{R}^4$ in the form of desired mass-normalized collective thrust and body rates.

We use a dense shaped reward formulation to learn the task of perception-aware autonomous drone racing. The reward r_t at time step t is given by

$$r_t = r_t^{\text{prog}} + r_t^{\text{perc}} + r_t^{\text{cmd}} - r_t^{\text{crash}} \tag{7}$$

in which r^{prog} rewards progress towards the next gate[35], r^{perc} encodes perception awareness by adjusting the attitude of the vehicle such that the optical axis of the camera points towards the centre of the next gate, r^{cmd} rewards smooth actions and r^{crash} is a binary penalty that is only active when colliding with a gate or when the platform leaves a predefined bounding box. If r^{crash} is triggered, the training episode ends.

Specifically, the reward terms are

$$\begin{aligned} r_t^{\text{prog}} &= \lambda_1 [d_{t-1}^{\text{Gate}} - d_t^{\text{Gate}}] \\ r_t^{\text{perc}} &= \lambda_2 \exp[\lambda_3 \cdot \delta_{\text{cam}}^4] \end{aligned} \tag{8}$$

$$r_t^{\text{cmd}} = \lambda_4 \mathbf{a}_t^{\omega} + \lambda_5 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 \tag{9}$$

$$r_t^{\text{crash}} = \begin{cases} 5.0, & \text{if } p_z < 0 \text{ or in collision with gate} \\ 0, & \text{otherwise} \end{cases}$$

in which d_t^{Gate} denotes the distance from the centre of mass of the vehicle to the centre of the next gate at time step t, δ_{cam} represents the angle between the optical axis of the camera and the centre of the next gate and $\mathbf{a}_t^{\omega}$ are the commanded body rates. The hyperparameters $\lambda_1, \ldots, \lambda_5$ balance different terms (Extended Data Table 1a).

Training details. Data collection is performed by simulating 100 agents in parallel that interact with the environment in episodes of 1,500 steps. At each environment reset, every agent is initialized at a random gate on the track, with bounded perturbation around a state previously observed when passing this gate. In contrast to previous work[44,49,50], we do not perform randomization of the platform dynamics at training time. Instead, we perform fine-tuning based on real-world data. The training environment is implemented using TensorFlow Agents[51]. The policy network and the value network are both represented by two-layer perceptrons with 128 nodes in each layer and LeakyReLU activations with a negative slope of 0.2. Network parameters are optimized using the Adam optimizer with learning rate 3×10^{-4} for both the policy network and the value network.

Policies are trained for a total of 1×10^8 environment interactions, which takes 50 min on a workstation (i9 12900K, RTX 3090, 32 GB RAM DDR5). Fine-tuning is performed for 2×10^7 environment interactions.

Residual model identification

We perform fine-tuning of the original policy based on a small amount of data collected in the real world. Specifically, we collect three full rollouts in the real world, corresponding to approximately 50 s of flight time. We fine-tune the policy by identifying residual observations and residual dynamics, which are then used for training in simulation. During this fine-tuning phase, only the weights of the control policy are updated, whereas the weights of the gate-detection network are kept constant.

Residual observation model. Navigating at high speeds results in substantial motion blur, which can lead to a loss of tracked visual features and severe drift in linear odometry estimates. We fine-tune policies with an odometry model that is identified from only a handful of trials recorded in the real world. To model the drift in odometry, we use Gaussian processes[36], as they allow fitting a posterior distribution of odometry perturbations, from which we can sample temporally consistent realizations.

Specifically, the Gaussian process model fits residual position, velocity and attitude as a function of the ground-truth robot state. The observation residuals are identified by comparing the observed visual–inertial odometry (VIO) estimates during a real-world rollout with the ground-truth platform states, which are obtained from an external motion-tracking system.

We treat each dimension of the observation separately, effectively fitting a set of nine 1D Gaussian processes to the observation residuals. We use a mixture of radial basis function kernels

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \sigma_{\text{f}}^2 \exp\left(-\frac{1}{2}(\mathbf{z}_i - \mathbf{z}_j)^{\top} L^{-2} (\mathbf{z}_i - \mathbf{z}_j)\right) + \sigma_{\text{n}}^2 \tag{10}$$

in which L is the diagonal length scale matrix and σ_{f} and σ_{n} represent the data and prior noise variance, respectively, and $\mathbf{z}_i$ and $\mathbf{z}_j$ represent data features. The kernel hyperparameters are optimized by maximizing the log marginal likelihood. After kernel hyperparameter optimization, we sample new realizations from the posterior distribution that are then used during fine-tuning of the policy. Extended Data Fig. 1 illustrates the residual observations in position, velocity and attitude in real-world rollouts, as well as 100 sampled realizations from the Gaussian process model.

Residual dynamics model. We use a residual model to complement the simulated robot dynamics[52]. Specifically, we identify residual accelerations as a function of the platform state $\mathbf{s}$ and the commanded mass-normalized collective thrust c:

从包含所有相关量测量值的 Betaflight 日志中识别。与低级控制器的模型一起，这使得模拟器能够正确地将集体推力和车身速率形式的动作转换为方程(1)中所需的电机速度Ω_{ss}。

政策培训

我们训练深度神经控制策略，以平台状态的形式直接映射观测值o_t，并以质量标准化集体推力和身体速率[44]的形式直接映射下一门观测值来控制动作u_t。控制策略是在模拟中使用无模型强化学习进行训练的。

训练算法。使用近端策略优化[31]进行训练。这种演员-批评家方法需要在训练期间联合优化两个神经网络：策略网络（将观察结果映射到行动）和价值网络（充当“批评家”并评估政策所采取的行动）。训练后，仅将策略网络部署在机器人上。

观察、行动和奖励。在时间 t 从环境获得的观测值 $o_t \in \mathbb{R}^{31}$ 包括：(1) 当前机器人状态的估计；(2) 下一个要通过的闸门在轨道布局上的相对位姿；(3) 上一步中应用的操作。具体来说，机器人状态的估计包含平台的位置、由旋转矩阵表示的速度和姿态，从而产生 $\mathbb{R}^{15}$ 中的向量。虽然模拟内部使用四元数，但我们使用旋转矩阵来表示态度以避免歧义[47]。通过提供四个门角相对于车辆的相对位置来编码下一个门的相对姿态，从而产生 $\mathbb{R}^{12}$ 中的向量。所有观察结果在传递到网络之前都经过标准化。由于价值网络仅在训练期间使用，因此它可以访问策略无法访问的有关环境的特权信息[48]。这些特权信息与策略网络的其他输入连接在一起，包含机器人的确切位置、方向和速度。

对于每个观察 o_t，策略网络会以期望的质量归一化集体推力和身体速率的形式产生一个动作 $\mathbb{R} a_t \in {}^4$。

我们使用密集形状的奖励公式来学习感知感知的自主无人机竞赛任务。时间步 t 的奖励 r_t 由下式给出

$$r_t = r_t^{\text{prog}} + r_t^{\text{perc}} + r_t^{\text{cmd}} - r_t^{\text{crash}} \tag{7}$$

其中r^{prog}奖励朝向下一个门的进展[35]，r^{perc}通过调整车辆的姿态来编码感知意识，使相机的光轴指向下一个门的中心，r^{cmd}奖励平滑的动作，r^{crash}是一个二元惩罚，仅在与门碰撞或平台离开预定义的边界框时才有效。如果触发 r^{crash}，则训练集结束。具体来说，奖励条件是

$$\begin{aligned} r_t^{\text{prog}} &= \lambda_1 [d_{t-1}^{\text{Gate}} - d_t^{\text{Gate}}] \\ r_t^{\text{perc}} &= \lambda_2 \exp[\lambda_3 \cdot \delta_{\text{cam}}^4] \end{aligned} \tag{8}$$

$$r_t^{\text{cmd}} = \lambda_4 \mathbf{a}_t^{\omega} + \lambda_5 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 \tag{9}$$

$$r_t^{\text{crash}} = \begin{cases} 5.0, \text{ if } p_z < 0 \text{ or in collision with gate} \\ 0, \quad \text{otherwise} \end{cases}$$

其中d_t^{Gate}表示时间步t时车辆质心到下一闸门中心的距离，δ_{cam}表示相机光轴与下一闸门中心之间的角度

和 a_t^{ω} 是命令的身体速率。超参数 $\lambda_1, \cdots, \lambda_5$ 平衡不同的项（扩展数据表 1a）。

培训细节。数据收集是通过并行模拟 100 个代理来执行的，这些代理以 1,500 个步骤的片段与环境进行交互。 在每次环境重置时，每个代理都会在轨道上的随机门处进行初始化，并在通过该门时围绕先前观察到的状态进行有界扰动。与之前的工作[44,49,50]相比，我们不在训练时对平台动态进行随机化。相反，我们根据真实数据进行微调。训练环境是使用 TensorFlow Agents[51] 实现的。策略网络和价值网络均由每层 128 个节点的两层感知器和负斜率为 0.2 的 LeakyReLU 激活表示。使用 Adam 优化器对策略网络和价值网络的网络参数进行优化，学习率为 3×10^{-4}。

策略总共针对 1×10^8 次环境交互进行训练，在工作站（i9 12900K、RTX 3090、32 GB RAM DDR5）上需要 50 分钟。针对 2×10^7 环境交互执行微调。

残差模型识别

我们根据现实世界中收集的少量数据对原始策略进行微调。具体来说，我们收集了现实世界中的三个完整部署，相当于大约 50 秒的飞行时间。我们通过识别残余观测值和残余动态来微调策略，然后将其用于模拟训练。在此微调阶段，仅更新控制策略的权重，而门检测网络的权重保持不变。

残差观察模型。高速导航会导致大量的运动模糊，这可能会导致跟踪视觉特征的丢失以及线性里程计估计的严重漂移。我们使用里程计模型来微调策略，该模型是从现实世界中记录的少数试验中确定的。为了对里程计中的漂移进行建模，我们使用高斯过程[36]，因为它们允许拟合里程计扰动的后验分布，从中我们可以采样时间一致的实现。

具体来说，高斯过程模型将残余位置、速度和姿态拟合为地面实况机器人状态的函数。 通过将现实世界推出期间观察到的视觉惯性里程计 (VIO) 估计值与从外部运动跟踪系统获得的地面实况平台状态进行比较来识别观测残差。

我们有效地单独处理观察的每个维度
将一组九个一维高斯过程拟合到观测残差。
我们使用径向基函数核的混合

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \sigma_{\text{f}}^2 \exp\left(-\frac{1}{2}(\mathbf{z}_i - \mathbf{z}_j)^{\top} L^{-2} (\mathbf{z}_i - \mathbf{z}_j)\right) + \sigma_{\text{n}}^2 \tag{10}$$

其中L 是对角线长度尺度矩阵，σ_{f}和σ_{n}分别表示数据和先验噪声方差，z_i和z_j表示数据特征。通过最大化对数边际似然来优化内核超参数。在内核超参数优化之后，我们从后验分布中采样新的实现，然后在策略的微调过程中使用这些新的实现。扩展数据图 1说明了现实世界中位置、速度和姿态的剩余观测值，以及高斯过程模型的 100 个采样实现。

残余动力学模型。我们使用残差模型来补充模拟的机器人动力学[52]。具体来说，我们将残余加速度确定为平台状态 s 和命令质量归一化集体推力 c 的函数：

$$\mathbf{a}_{\text{res}} = \text{KNN}(\mathbf{s}, c) \tag{11}$$

We use k-nearest neighbour regression with $k = 5$. The size of the dataset used for residual dynamics model identification depends on the track layout and ranges between 800 and 1,000 samples for the track layout used in this work.

Gate detection

To correct for drift accumulated by the VIO pipeline, the gates are used as distinct landmarks for relative localization. Specifically, gates are detected in the onboard camera view by segmenting gate corners[26]. The greyscale images provided by the Intel RealSense Tracking Camera T265 are used as input images for the gate detector. The architecture of the segmentation network is a six-level U-Net[53] with (8, 16, 16, 16, 16, 16) convolutional filters of size (3, 3, 3, 5, 7, 7) per level and a final extra layer operating on the output of the U-Net containing 12 filters. As the activation function, LeakyReLU with $\alpha = 0.01$ is used. For deployment on the NVIDIA Jetson TX2, the network is ported to TensorRT. To optimize memory footprint and computation time, inference is performed in half-precision mode (FP16) and images are downsampled to size 384 × 384 before being fed to the network. One forward pass through the network takes 40 ms on the NVIDIA Jetson TX2.

VIO drift estimation

The odometry estimates from the VIO pipeline[54] exhibit substantial drift during high-speed flight. We use gate detection to stabilize the pose estimates produced by VIO. The gate detector outputs the coordinates of the corners of all visible gates. A relative pose is first estimated for all predicted gates using infinitesimal plane-based pose estimation (IPPE)[34]. Given this relative pose estimate, each gate observation is assigned to the closest gate in the known track layout, thus yielding a pose estimate for the drone.

Owing to the low frequency of the gate detections and the high quality of the VIO orientation estimate, we only refine the translational components of the VIO measurements. We estimate and correct for the drift of the VIO pipeline using a Kalman filter that estimates the translational drift $\mathbf{p}_{\text{d}}$ (position offset) and its derivative, the drift velocity $\mathbf{v}_{\text{d}}$. The drift correction is performed by subtracting the estimated drift states $\mathbf{p}_{\text{d}}$ and $\mathbf{v}_{\text{d}}$ from the corresponding VIO estimates. The Kalman filter state $\mathbf{x}$ is given by $\mathbf{x} = [\mathbf{p}_{\text{d}}^{\top}, \mathbf{v}_{\text{d}}^{\top}]^{\top} \in \mathbb{R}^6$.

The state $\mathbf{x}$ and covariance P updates are given by:

$$\mathbf{x}_{k+1} = F\mathbf{x}_k, \quad P_{k+1} = FP_kF^{\top} + Q, \tag{12}$$

$$F = \begin{bmatrix} \mathbb{I}^{3\times3} & \mathrm{d}t\,\mathbb{I}^{3\times3} \\ 0^{3\times3} & \mathbb{I}^{3\times3} \end{bmatrix}, \quad Q = \begin{bmatrix} \sigma_{\text{pos}}\mathbb{I}^{3\times3} & 0^{3\times3} \\ 0^{3\times3} & \sigma_{\text{vel}}\mathbb{I}^{3\times3} \end{bmatrix}. \tag{13}$$

On the basis of measurements, the process noise is set to $\sigma_{\text{pos}} = 0.05$ and $\sigma_{\text{vel}} = 0.1$. The filter state and covariance are initialized to zero. For each measurement $\mathbf{z}_k$ (pose estimate from a gate detection), the predicted VIO drift $\mathbf{x}_k^-$ is corrected to the estimate $\mathbf{x}_k^+$ according to the Kalman filter equations:

$$\begin{aligned} K_k &= P_k^- H_k^{\top}(H_k P_k^- H_k^{\top} + R)^{-1}, \\ \mathbf{x}_k^+ &= \mathbf{x}_k^- + K_k(\mathbf{z}_k - H(\mathbf{x}_k^-)), \\ P_k^+ &= (I - K_k H_k)P_k^-, \end{aligned} \tag{14}$$

in which K_k is the Kalman gain, R is the measurement covariance and H_k is the measurement matrix. If several gates have been detected in a single camera frame, all relative pose estimates are stacked and processed in the same Kalman filter update step. The main source of measurement error is the uncertainty in the gate-corner detection of the network. This error in the image plane results in a pose error when IPPE is applied. We opted for a sampling-based approach to estimate the pose error from the known average gate-corner-detection uncertainty. For each gate, the IPPE algorithm is applied to the nominal gate observation as well as to 20 perturbed gate-corner estimates. The resulting distribution of pose estimates is then used to approximate the measurement covariance R of the gate observation.

Simulation results

Reaching champion-level performance in autonomous drone racing requires overcoming two challenges: imperfect perception and incomplete models of the system's dynamics. In controlled experiments in simulation, we assess the robustness of our approach to both of these challenges. To this end, we evaluate performance in a racing task when deployed in four different settings. In setting (1), we simulate a simplistic quadrotor model with access to ground-truth state observations. In setting (2), we replace the ground-truth state observations with noisy observations identified from real-world flights. These noisy observations are generated by sampling one realization from the residual observation model and are independent of the perception awareness of the deployed controller. Settings (3) and (4) share the observation models with the previous two settings, respectively, but replace the simplistic dynamics model with more accurate aerodynamical simulation[43]. These four settings allow controlled assessment of the sensitivity of the approach to changes in the dynamics and the observation fidelity.

In all four settings, we benchmark our approach against the following baselines: zero-shot, domain randomization and time-optimal. The zero-shot baseline represents a learning-based racing policy[35] trained using model-free RL that is deployed zero-shot from the training domain to the test domain. The training domain of the policy is equal to experimental setting (1), that is, idealized dynamics and ground-truth observations. Domain randomization extends the learning strategy from the zero-shot baseline by randomizing observations and dynamics properties to increase robustness. The time-optimal baseline uses a precomputed time-optimal trajectory[28] that is tracked using an MPC controller. This approach has shown the best performance in comparison with other model-based methods for time-optimal flight[55,56]. The dynamics model used by the trajectory generation and the MPC controller matches the simulated dynamics of experimental setting (1).

Performance is assessed by evaluating the fastest lap time, the average and minimum observed gate margin of successfully passed gates and the percentage of track successfully completed. The gate margin metric measures the distance between the drone and the closest point on the gate when crossing the gate plane. A high gate margin indicates that the quadrotor passed close to the centre of the gate. Leaving a smaller gate margin can increase speed but can also increase the risk of collision or missing the gate. Any lap that results in a crash is not considered valid.

The results are summarized in Extended Data Table 1c. All approaches manage to successfully complete the task when deployed in idealized dynamics and ground-truth observations, with the time-optimal baseline yielding the lowest lap time. When deployed in settings that feature domain shift, either in the dynamics or the observations, the performance of all baselines collapses and none of the three baselines are able to complete even a single lap. This performance drop is exhibited by both learning-based and traditional approaches. By contrast, our approach, which features empirical models of dynamics and observation noise, succeeds in all deployment settings, with small increases in lap time.

The key feature that enables our approach to succeed across deployment regimes is the use of an empirical model of dynamics and observation noise, estimated from real-world data. A comparison between an approach that has access to such data and approaches that do not is not entirely fair. For that reason, we also benchmark the performance of all baseline approaches when having access to the same real-world data used by our approach. Specifically, we

$$\mathbf{a}_{\text{res}} = \text{KNN}(\mathbf{s}, c) \tag{11}$$

我们使用 k-最近邻回归与 $k = 5$。用于残余动力学模型识别的数据集的大小取决于轨道布局，并且本工作中使用的轨道布局的样本范围在 800 到 1,000 个样本之间。

闸门检测

为了纠正 VIO 管道累积的漂移，门被用作相对定位的独特标志。具体来说，通过分割门角[26]，在板载摄像头视图中检测门。 英特尔实感跟踪摄像头 T265 提供的灰度图像用作门检测器的输入图像。分割网络的架构是一个六级 U-Net[53]，每级具有 (8, 16, 16, 16, 16, 16) 个大小为 (3, 3, 3, 5, 7, 7) 的卷积滤波器，最后一个额外层对 U-Net 的输出进行操作包含 12 个过滤器。使用 $\alpha = 0.01$ 的 LeakyReLU 作为激活函数。为了在 NVIDIA Jetson TX2 上部署，网络被移植到 TensorRT。为了优化内存占用和计算时间，推理在半精度模式（FP16）下执行，图像在输入网络之前被下采样到大小 384×384。在 NVIDIA Jetson TX2 上，通过网络的一次前向传递需要 40 ms。

VIO漂移估计

VIO 管道[54] 的里程计估计在高速飞行期间表现出很大的漂移。我们使用门检测来稳定 VIO 生成的姿态估计。门检测器输出所有可见门的角点坐标。首先使用基于无穷小平面的姿态估计 (IPPE)[34] 估计所有预测门的相对姿态。给定这个相对位姿估计，每个门观察被分配给已知轨道布局中最近的门，从而产生无人机的位姿估计。

由于门检测的频率较低和 VIO 方向估计的高质量，我们仅细化 VIO 测量的平移分量。我们使用卡尔曼滤波器来估计和校正 VIO 管道的漂移，该滤波器估计平移漂移 p_d（位置偏移）及其导数，即漂移速度 v_d。通过从相应的 VIO 估计值中减去估计的漂移状态 p_d 和 v_d 来执行漂移校正。卡尔曼滤波器状态 x 由以下公式给出： ℝ x p v ⊤ ⊤⊤ [,] ∈

状态 x 和协方差 P 更新由下式给出：

$$\mathbf{x}_{k+1} = F\mathbf{x}_k, \quad P_{k+1} = FP_kF^\top + Q, \tag{12}$$

$$F = \begin{bmatrix} \mathbb{I}^{3\times3} & \text{d}t\,\mathbb{I}^{3\times3} \\ 0^{3\times3} & \mathbb{I}^{3\times3} \end{bmatrix}, \quad Q = \begin{bmatrix} \sigma_{\text{pos}}\mathbb{I}^{3\times3} & 0^{3\times3} \\ 0^{3\times3} & \sigma_{\text{vel}}\mathbb{I}^{3\times3} \end{bmatrix}. \tag{13}$$

根据测量结果，过程噪声设置为 $\sigma_{\text{pos}} = 0.05$ 和 $\sigma_{\text{vel}} = 0.1$。滤波器状态和协方差被初始化为零。对于每个测量 z_k（来自门检测的姿态估计），根据卡尔曼滤波器方程将预测的 VIO 漂移 x_k^- 校正为估计值 x_k^+：

$$\begin{aligned} K_k &= P_k^- H_k^\top (H_k P_k^- H_k^\top + R)^{-1}, \\ \mathbf{x}_k^+ &= \mathbf{x}_k^- + K_k(\mathbf{z}_k - H(\mathbf{x}_k^-)), \\ P_k^+ &= (I - K_k H_k) P_k^-, \end{aligned} \tag{14}$$

其中 K_k 是卡尔曼增益，R 是测量协方差，H_k 是测量矩阵。如果在单个相机帧中检测到多个门，则所有相对姿态估计都将在同一卡尔曼滤波器更新步骤中堆叠和处理。测量误差的主要来源是网络门角检测的不确定性。应用 IPPE 时，图像平面中的此误差会导致位姿误差。

我们选择基于采样的方法来根据已知的平均门角检测不确定性来估计位姿误差。对于每个门，IPPE 算法应用于标称门观察以及 20 个扰动门角估计。然后使用所得的姿态估计分布来近似门观测的测量协方差 $\boldsymbol{R}$。

模拟结果

在自主无人机竞赛中达到冠军水平需要克服两个挑战：不完美的感知和不完整的系统动力学模型。在模拟受控实验中，我们评估了我们的方法应对这两个挑战的稳健性。为此，我们评估了在四种不同设置中部署时的赛车任务性能。在设置（1）中，我们模拟了一个简单的四旋翼飞行器模型，可以访问地面真实状态观测结果。在设置（2）中，我们用从现实世界飞行中识别出的噪声观测值替换了地面实况观测值。这些噪声观测是通过从残差观测模型中采样一种实现而生成的，并且独立于所部署控制器的感知意识。设置（3）和（4）分别与前两个设置共享观测模型，但用更精确的空气动力学模拟代替简单的动力学模型[43]。这四种设置允许对方法对动力学变化和观察保真度变化的敏感性进行受控评估。

在所有四种设置中，我们根据以下基线对我们的方法进行基准测试：零样本、域随机化和时间最优。 零样本基线表示使用无模型强化学习训练的基于学习的赛车策略[35]，该策略从训练域零样本部署到测试域。策略的训练域等于实验设置（1），即理想化动态和地面实况观察。域随机化通过随机化观察和动力学属性来提高鲁棒性，从而将学习策略从零样本基线扩展到了零样本基线。时间最佳基线使用预先计算的时间最佳轨迹[28]，并使用 MPC 控制器进行跟踪。与其他基于模型的时间最佳飞行方法相比，该方法显示出最佳性能[55,56]。 轨迹生成和 MPC 控制器使用的动力学模型与实验设置 (1) 的模拟动力学相匹配。

通过评估最快单圈时间、成功通过闸门的平均和最小观测闸门裕度以及成功完成赛道的百分比来评估性能。门边距指标测量无人机穿过门平面时与门上最近点之间的距离。高门边距表明四旋翼飞行器靠近门中心通过。留较小的浇口余量可以提高速度，但也会增加碰撞或错过浇口的风险。任何导致碰撞的单圈均不被视为有效。

结果总结在扩展数据表 1c中。当部署在理想化动力学和地面实况观测中时，所有方法都能成功完成任务，并且时间最佳基线产生最短的单圈时间。当部署在具有域转移特征的设置中时，无论是在动力学还是在观测中，所有基线的性能都会崩溃，并且三个基线都无法完成哪怕是一圈。基于学习的方法和传统方法都表现出这种性能下降。相比之下，我们的方法以动力学和观察噪声的经验模型为特色，在所有部署设置中都取得了成功，单圈时间略有增加。

使我们的方法能够在各种部署方案中取得成功的关键特征是使用根据现实世界数据估计的动态和观察噪声的经验模型。对能够访问此类数据的方法和不能访问此类数据的方法进行比较并不完全公平。因此，在访问我们的方法使用的相同真实世界数据时，我们还会对所有基线方法的性能进行基准测试。具体来说，我们

compare the performance in experimental setting (2), which features the idealized dynamics model but noisy perception. All baseline approaches are provided with the predictions of the same Gaussian process model that we use to characterize observation noise. The results are summarized in Extended Data Table 1b. All baselines benefit from the more realistic observations, yielding higher completion rates. Nevertheless, our approach is the only one that reliably completes the entire track. As well as the predictions of the observation noise model, our approach also takes into account the uncertainty of the model. For an in-depth comparison of the performance of RL versus optimal control in controlled experiments, we refer the reader to ref. 57.

Fine-tuning for several iterations

We investigate the extent of variations in behaviour across iterations. The findings of our analysis reveal that subsequent fine-tuning operations result in negligible enhancements in performance and alterations in behaviour (Extended Data Fig. 2).

In the following, we provide more details on this investigation. We start by enumerating the fine-tuning steps to provide the necessary notation:

1. Train policy-0 in simulation.
2. Deploy policy-0 in the real world. The policy operates on ground-truth data from a motion-capture system.
3. Identify residuals observed by policy-0 in the real world.
4. Train policy-1 by fine-tuning policy-0 on the identified residuals.
5. Deploy policy-1 in the real world. The policy operates only on onboard sensory measurements.
6. Identify residuals observed by policy-1 in the real world.
7. Train policy-2 by fine-tuning policy-1 on the identified residuals.

We compare the performance of policy-1 and policy-2 in simulation after fine-tuning on their respective residuals. The results are illustrated in Extended Data Fig. 2. We observe that the difference in distance from gate centres, which is a metric for the safety of the policy, is 0.09 ± 0.08 m. Furthermore, the difference in the time taken to complete a single lap is 0.02 ± 0.02 s. Note that this lap-time difference is substantially smaller than the difference between the single-lap completion times of Swift and the human pilots (0.16 s).

Drone hardware configuration

The quadrotors used by the human pilots and Swift have the same weight, shape and propulsion. The platform design is based on the Agilicious framework[58]. Each vehicle has a weight of 870 g and can produce a maximum static thrust of approximately 35 N, which results in a static thrust-to-weight ratio of 4.1. The base of each platform consists of an Armattan Chameleon 6″ main frame that is equipped with T-Motor Velox 2306 motors and 5″, three-bladed propellers. An NVIDIA Jetson TX2 accompanied by a Connect Tech Quasar carrier board provides the main compute resource for the autonomous drones, featuring a six-core CPU running at 2 GHz and a dedicated GPU with 256 CUDA cores running at 1.3 GHz. Although forward passes of the gate-detection network are performed on the GPU, the racing policy is evaluated on the CPU, with one inference pass taking 8 ms. The autonomous drones carry an Intel RealSense Tracking Camera T265 that provides VIO estimates[59] at 100 Hz that are fed by USB to the NVIDIA Jetson TX2. The human-piloted drones carry neither a Jetson computer nor a RealSense camera and are instead equipped with a corresponding ballast weight. Control commands in the form of collective thrust and body rates produced by the human pilots or Swift are sent to a commercial flight controller, which runs on an STM32 processor operating at 216 MHz. The flight controller is running Betaflight, an open-source flight-control software[45].

Human pilot impressions

The following quotes convey the impressions of the three human champions who raced against Swift.

Alex Vanover:

- These races will be decided at the split S, it is the most challenging part of the track.
- This was the best race! I was so close to the autonomous drone, I could really feel the turbulence when trying to keep up with it.

Thomas Bitmatta:

- The possibilities are endless, this is the start of something that could change the whole world. On the flip side, I'm a racer, I don't want anything to be faster than me.
- As you fly faster, you trade off precision for speed.
- It's inspiring to see the potential of what drones are actually capable of. Soon, the AI drone could even be used as a training tool to understand what would be possible.

Marvin Schaepper:

- It feels different racing against a machine, because you know that the machine doesn't get tired.

Research ethics

The study has been conducted in accordance with the Declaration of Helsinki. The study protocol is exempt from review by an ethics committee according to the rules and regulations of the University of Zurich, because no health-related data has been collected. The participants gave their written informed consent before participating in the study.

Data availability

All (other) data needed to evaluate the conclusions in the paper are present in the paper or the extended data. Motion-capture recordings of the race events with accompanying analysis code can be found in the file 'racing_data.zip' on Zenodo at https://doi.org/10.5281/zenodo.7955278.

Code availability

Pseudocode for Swift detailing the training process and algorithms can be found in the file 'pseudocode.zip' on Zenodo at https://doi.org/10.5281/zenodo.7955278. To safeguard against potential misuse, the full source code associated with this research will not be made publicly available.

43. Bauersfeld, L., Kaufmann, E., Foehn, P., Sun, S. & Scaramuzza, D. in *Proc. Robotics: Science and Systems XVII* 42 (Robotics: Science and Systems Foundation, 2021).
44. Kaufmann, E., Bauersfeld, L. & Scaramuzza, D. in *Proc. 2022 International Conference on Robotics and Automation (ICRA)* 10504–10510 (IEEE, 2022).
45. The Betaflight Open Source Flight Controller Firmware Project. Betaflight. https://github.com/betaflight/betaflight (2022).
46. Bauersfeld, L. & Scaramuzza, D. Range, endurance, and optimal speed estimates for multicopters. *IEEE Robot. Autom. Lett.* **7**, 2953–2960 (2022).
47. Zhou, Y., Barnes, C., Lu, J., Yang, J. & Li, H. in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5745–5753 (IEEE, 2019).
48. Pinto, L., Andrychowicz, M., Welinder, P., Zaremba, W. & Abbeel, P. in *Proc. Robotics: Science and Systems XIV* (MIT Press Journals, 2018).
49. Molchanov, A. et al. in *Proc. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 59–66 (IEEE, 2019).
50. Andrychowicz, O. M. et al. Learning dexterous in-hand manipulation. *Int. J. Robot. Res.* **39**, 3–20 (2020).
51. Guadarrama, S. et al. TF-Agents: a library for reinforcement learning in TensorFlow. https://github.com/tensorflow/agents (2018).
52. Torrente, G., Kaufmann, E., Foehn, P. & Scaramuzza, D. Data-driven MPC for quadrotors. *IEEE Robot. Autom. Lett.* **6**, 3769–3776 (2021).
53. Ronneberger, O., Fischer, P. & Brox, T. in *Proc. International Conference on Medical Image Computing and Computer-assisted Intervention* 234–241 (Springer, 2015).
54. Intel RealSense T265 series product family. https://www.intelrealsense.com/wp-content/uploads/2019/09/Intel_RealSense_Tracking_Camera_Datasheet_Rev004_release.pdf (2019).
55. Ryou, G., Tal, E. & Karaman, S. Multi-fidelity black-box optimization for time-optimal quadrotor maneuvers. *Int. J. Robot. Res.* **40**, 1352–1369 (2021).
56. Pham, H. & Pham, Q.-C. A new approach to time-optimal path parameterization based on reachability analysis. *IEEE Trans. Robot.* **34**, 645–659 (2018).
57. Song, Y., Romero, A., Müller, M., Koltun, V. & Scaramuzza, D. Reaching the limit in autonomous racing: optimal control versus reinforcement learning. *Sci. Robot.* (in the press).

比较实验设置（2）中的性能，该设置具有理想化动力学模型但存在噪声感知。所有基线方法都提供了我们用来表征观测噪声的相同高斯过程模型的预测。结果总结在扩展数据表 1b 中。所有基线都受益于更现实的观察，产生更高的完成率。尽管如此，我们的方法是唯一能够可靠地完成整个赛道的方法。除了观测噪声模型的预测之外，我们的方法还考虑了模型的不确定性。为了在受控实验中深入比较强化学习与最优控制的性能，我们建议读者参考参考文献 57。

多次迭代微调

我们研究迭代过程中行为的变化程度。 我们的分析结果表明，后续的微调操作对性能的提高和行为的改变几乎可以忽略不计（扩展数据图 2）。

下面，我们将提供有关此调查的更多详细信息。我们首先列举微调步骤以提供必要的符号：

1. 在模拟中训练policy-0。 2. 在现实世界中部署policy-0。该策略基于来自动作捕捉系统的地面实况数据。 3. 识别现实世界中policy-0观察到的残差。 4. 通过针对已识别的残差微调策略 0 来训练策略 1。 5. 在现实世界中部署策略 1。该政策仅适用于船上感官测量。 6. 识别现实世界中政策 1 观察到的残差。 7. 通过根据已识别的残差微调策略 1 来训练策略 2。

我们在对各自的残差进行微调后，在模拟中比较了策略 1 和策略 2 的性能。结果如扩展数据图 2所示。我们观察到，与门中心的距离差异（衡量策略安全性的指标）为 0.09 ± 0.08 m。此外，完成一圈所需的时间差异为 0.02 ± 0.02 s。请注意，这个单圈时间差异远小于 Swift 和人类飞行员的单圈完成时间之间的差异（0.16 s）。

无人机硬件配置

人类飞行员和斯威夫特使用的四旋翼飞行器具有相同的重量、形状和推进力。平台设计基于Agilicious框架[58]。每辆车的重量为 870 g，可产生约 35 N 的最大静态推力，这导致静态推力重量比为 4.1。每个平台的底座均由 Armattan Chameleon 6 英寸主框架组成，该主框架配备 T-Motor Velox 2306 电机和 5 英寸三叶螺旋桨。 NVIDIA Jetson TX2 搭配 Connect Tech Quasar 载板为自主无人机提供主要计算资源，具有运行频率为 2 GHz 的六核 CPU 和运行频率为 1.3 GHz 的具有 256 个 CUDA 内核的专用 GPU。尽管门检测网络的前向传递是在 GPU 上执行的，但竞赛策略是在 CPU 上评估的，一次推理传递需要 8 ms。自动无人机搭载英特尔实感跟踪摄像头 T265，可提供 100 Hz 频率下的 VIO 估计值[59]，并通过 USB 馈送到 NVIDIA Jetson TX2。人类驾驶的无人机既不携带 Jetson 计算机，也不携带 RealSense 摄像头，而是配备了相应的压载物。 人类飞行员或 Swift 产生的集体推力和机身速率形式的控制命令被发送到商用飞行控制器，该控制器在运行频率为 216 MHz 的 STM32 处理器上运行。 飞行控制器运行 Betaflight，一种开源飞行控制软件[45]。

人类飞行员印象

以下引用传达了与斯威夫特比赛的三位人类冠军的印象。

亚历克斯·范诺弗：

- 这些比赛将在Split S赛段决出胜负，这是赛道上最具挑战性的部分。
- 这是最好的比赛！我距离自动无人机非常近，当我试图跟上它时，我真的能感觉到湍流。

托马斯·比特马塔：

- 可能性是无限的，这是可能改变整个世界的事情的开始。另一方面，我是一名赛车手，我不希望任何东西比我更快。
- 当你飞得更快时，你需要牺牲精度来换取速度。
- 看到无人机的实际潜力真是令人鼓舞。很快，人工智能无人机甚至可以用作训练工具，以了解可能发生的事情。

马文·谢珀：

- 与机器比赛的感觉不同，因为你知道机器不累。

研究伦理

该研究是根据赫尔辛基宣言进行的。根据苏黎世大学的规章制度，该研究方案免于伦理委员会的审查，因为没有收集与健康相关的数据。参与者在参与研究之前签署了书面知情同意书。

数据可用性

评估论文中的结论所需的所有（其他）数据都存在于论文或扩展数据中。比赛事件的动作捕捉记录以及随附的分析代码可以在 Zenodo 上的文件“racing_data.zip”中找到，网址为 https://doi.org/10.5281/zenodo.7955278。

代码可用性

详细介绍训练过程和算法的 Swift 伪代码可以在 Zenodo 上的文件“pseudocode.zip”中找到，网址为 https://doi。 org/10.5281/zenodo.7955278。为了防止潜在的滥用，与本研究相关的完整源代码将不会公开。

43. Bauersfeld, L.、Kaufmann, E.、Foehn, P.、Sun, S. 和 Scaramuzza, D.，参见 *Proc. Robotics: Science and Systems XVII* 42（机器人：科学与系统基金会，2021 年）。 44. Kaufmann, E.、Bauersfeld, L. 和 Scaramuzza, D.，参见 *Proc. 2022 International Conference on Robotics and Automation (ICRA)* 10504–10510（IEEE，2022）。 45. Betaflight 开源飞行控制器固件项目。贝塔飞行。 https://github。 com/betaflight/betaflight (2022)。 46. Bauersfeld, L. 和 Scaramuzza, D. 多旋翼飞行器的航程、耐力和最佳速度估计。 *IEEE Robot. Autom. Lett.* 7, 2953–2960 (2022)。 47. Zhou, Y.、Barnes, C.、Lu, J.、Yang, J. 和 Li, H.，参见 *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5745–5753（IEEE，2019）。 48. Pinto, L.、Andrychowicz, M.、Welinder, P.、Zaremba, W. 和 Abbeel, P.，见 *Proc. Robotics: Science and Systems XIV*（麻省理工学院新闻期刊，2018 年）。 49. Molchanov，A. 等人。在 *Proc. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* 59–66（IEEE，2019）中。 50. Andrychowicz, O. M. 等人。学习灵巧的手部操作。 *Int. J. Robot. Res.* 39, 3–20 (2020)。 51.Guadarrama, S. 等人。 TF-Agents：TensorFlow 中的强化学习库。 https://github.com/tensorflow/agents（2018）。 52. Torrente, G.、Kaufmann, E.、Foehn, P. 和 Scaramuzza, D. 四旋翼飞行器的数据驱动 MPC。 *IEEE Robot. Autom. Lett.* 6, 3769–3776 (2021)。 53. Ronneberger, O.、Fischer, P. 和 Brox, T. in *Proc. International Conference on Medical Image Computing and Computer-assisted Intervention* 234–241 (Springer, 2015)。 54.英特尔实感T265系列产品家族。 https://www.intelrealsense.com/wp-content/uploads/2019/09/Intel_RealSense_Tracking_Camera_Datasheet_Rev004_release.pdf (2019)。 55. Ryou, G.、Tal, E. 和 Karaman, S. 用于时间最佳四旋翼飞行器操纵的多保真度黑盒优化。 *Int. J. Robot. Res.* 40, 1352–1369 (2021)。 56. Pham, H. 和 Pham, Q.-C。基于可达性分析的时间最优路径参数化新方法*IEEE Trans. Robot.* 34, 645–659 (2018)。 57. Song, Y.、Romero, A.、Müller, M.、Koltun, V. 和 Scaramuzza, D. 达到自动驾驶赛车的极限：最优控制与强化学习。 *Sci. Robot.*（在媒体上）。

58. Foehn, P. et al. Agilicious: open-source and open-hardware agile quadrotor for vision-based flight. *Sci. Robot.* **7**, eabl6259 (2022).
59. Jones, E. S. & Soatto, S. Visual-inertial navigation, mapping and localization: a scalable real-time causal approach. *Int. J. Robot. Res.* **30**, 407–430 (2011).

Acknowledgements The authors thank A. Vanover, T. Bitmatta and M. Schaepper for accepting to race against Swift. The authors also thank C. Pfeiffer, T. Längle and A. Barden for their contributions to the organization of the race events and the drone hardware design. This work was supported by Intel's Embodied AI Lab, the Swiss National Science Foundation (SNSF) through the National Centre of Competence in Research (NCCR) Robotics and the European Research Council (ERC) under grant agreement 864042 (AGILEFLIGHT).

Author contributions E.K. formulated the main ideas, implemented the system, performed the experiments and data analysis and wrote the paper. L.B. contributed to the main ideas, the experiments, data analysis, paper writing and designed the graphical illustrations. A.L. formulated the main ideas and contributed to the experimental design, data analysis and paper writing. M.M. contributed to the experimental design, data analysis and paper writing. V.K. contributed to the main ideas, the experimental design, the analysis of experiments and paper writing. D.S. contributed to the main ideas, experimental design, analysis of experiments, paper writing and provided funding.

Competing interests The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41586-023-06419-4.

Correspondence and requests for materials should be addressed to Elia Kaufmann.

Peer review information *Nature* thanks Sunggoo Jung, Karime Pereida and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. Peer reviewer reports are available.

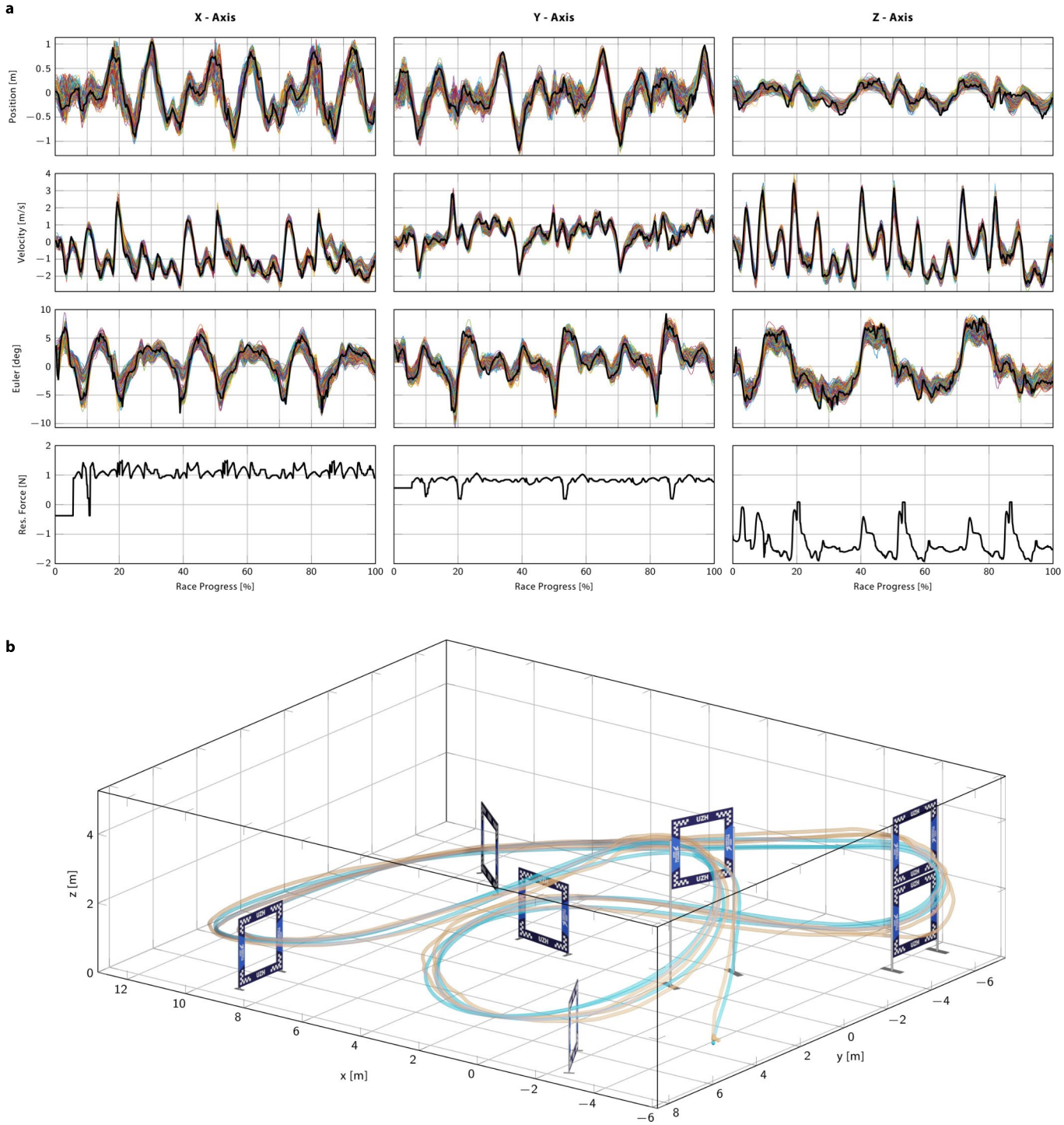
58.Foehn, P. 等人。 Agilicious：用于基于视觉的飞行的开源和开放硬件敏捷四旋翼飞行器。 *Sci. Robot.* 7，eabl6259 (2022)。 59. Jones, E. S. & Soatto, S. 视觉惯性导航、绘图和定位：一种可扩展的实时因果方法。 *Int. J. Robot. Res.* 30, 407–430 (2011)。

致谢 作者感谢 A. Vanover、T. Bitmatta 和 M. Schaepper 接受
与斯威夫特比赛。作者还感谢 C. Pfeiffer、T. Längle 和 A. Barden 对赛事组织和无人机硬件设计的贡献。这项工作得到了英特尔嵌入式人工智能实验室、瑞士国家科学基金会 (SNSF) 通过国家机器人研究能力中心 (NCCR) 和欧洲研究理事会 (ERC) 的资助协议 864042 (AGILEFLIGHT) 的支持。

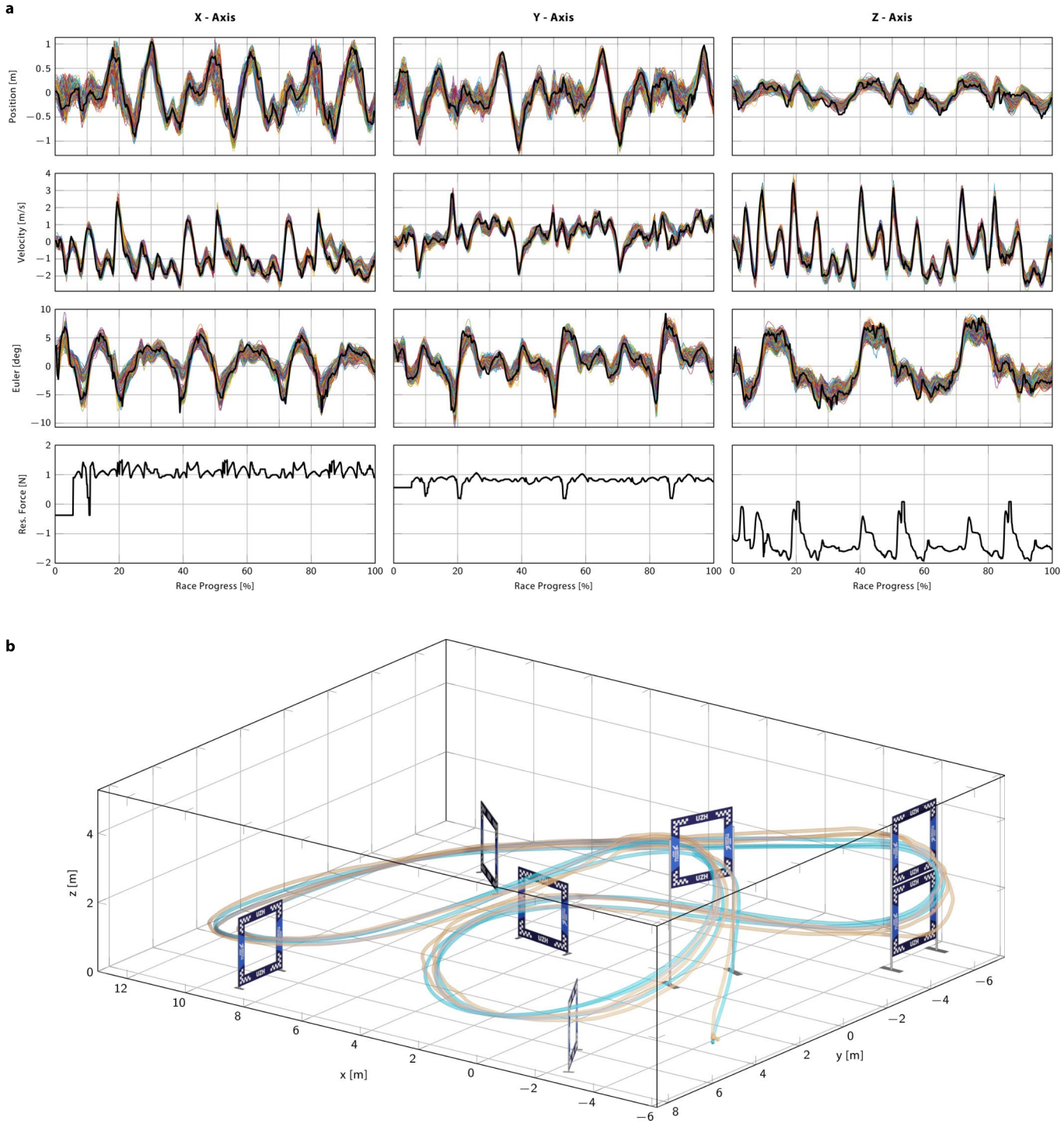
作者贡献 E.K.制定主要思想，实现系统，进行实验和数据分析并撰写论文。磅。贡献了主要思想、实验、数据分析、论文写作并设计了图形插图。 A.L. 阐述了主要思想，并为实验设计、数据分析和论文写作做出了贡献。毫米。为实验设计、数据分析和论文写作做出了贡献。 V.K.贡献了主要思想、实验设计、实验分析和论文写作。 D.S. 为主要思想、实验设计、实验分析、论文写作做出了贡献并提供了资金。

竞争利益 作者声明不存在竞争利益。

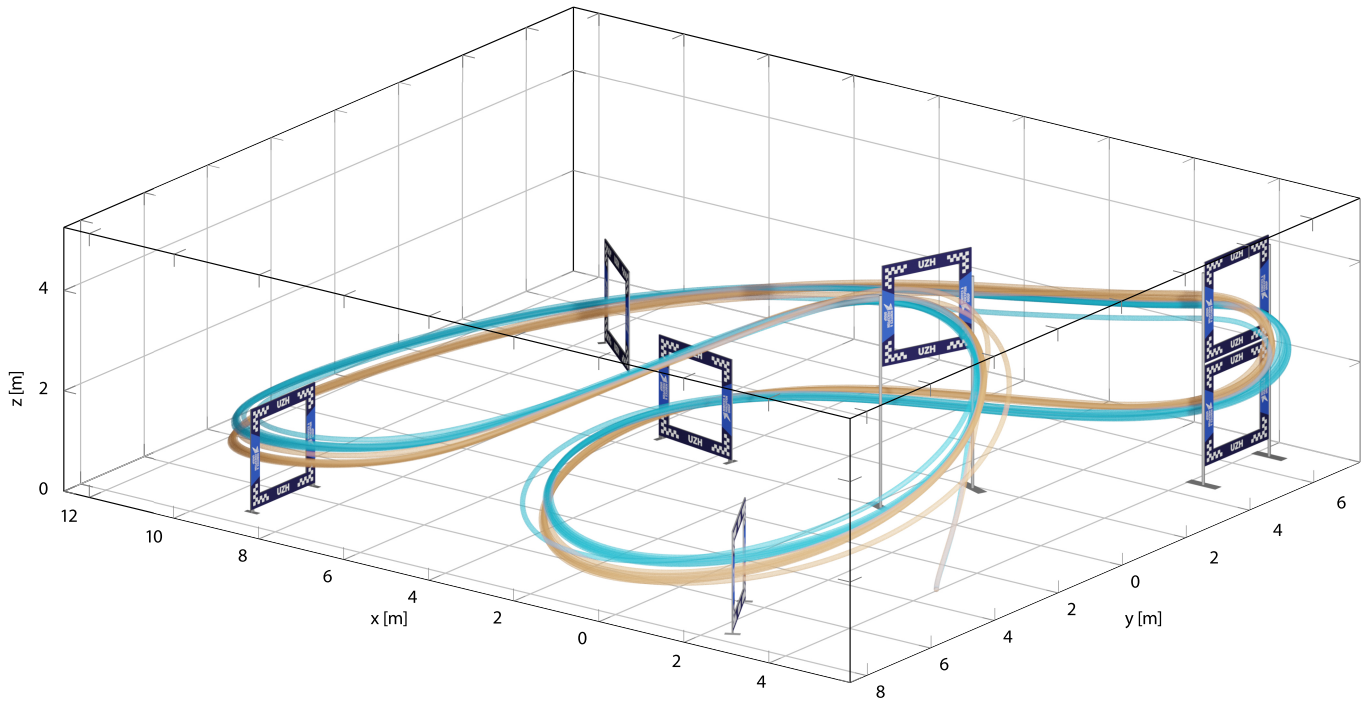
附加信息
补充信息 在线版本包含补充材料，可在以下位置获取：
https://doi.org/10.1038/s41586-023-06419-4。信件和材料请求应发送给 Elia Kaufmann。同行评审信息 *Nature* 感谢 Sunggoo Jung、Karime Pereida 和其他匿名审稿人对这项工作的同行评审所做的贡献。同行评审报告可供使用。重印和许可信息可在 http://www.nature.com/reprints 上找到。



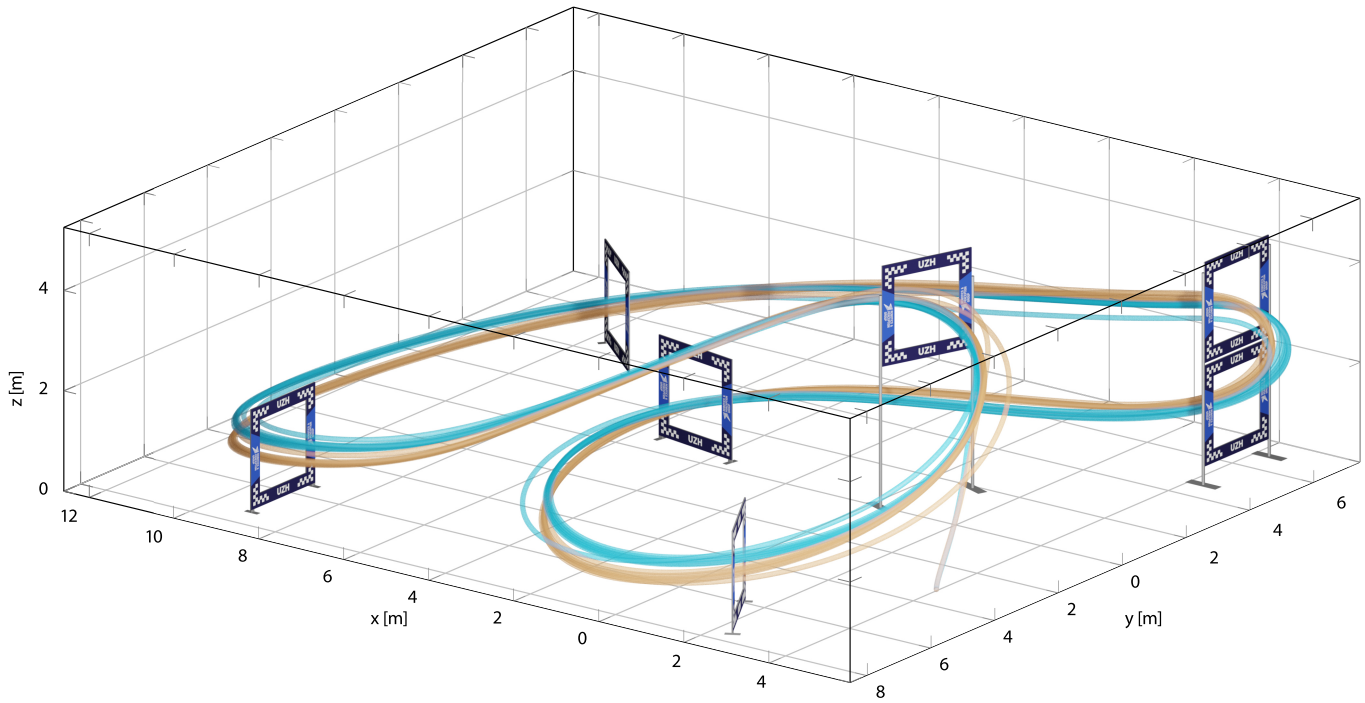
Extended Data Fig. 1 | Residual models. a, Visualization of the residual observation model and the residual dynamics model identified from real-world data. Black curves depict the residual observed in the real world and coloured lines show 100 sampled realizations of the residual observation model. Each plot depicts an entire race, that is, three laps. **b**, Predicted residual observation for a simulated rollout. Blue, ground-truth position provided by the simulator; orange, perturbed position generated by the Gaussian process residual.



扩展数据图 1 | 残差模型。 a，从真实世界数据中识别的残差观测模型和残差动力学模型的可视化。黑色曲线描绘了在现实世界中观察到的残差，彩色线显示了残差观察模型的 100 个采样实现。每个图描绘了整个比赛，即三圈。 b，模拟推出的预测剩余观测值。模拟器提供的蓝色真实位置； 橙色，由高斯过程残差生成的扰动位置。



Extended Data Fig. 2 | Multi-iteration fine-tuning. Rollout comparison after fine-tuning the policy for one iteration (blue) and two iterations (orange).



Extended Data Fig. 2 | 多次迭代微调。对一次迭代（蓝色）和两次迭代（橙色）的策略进行微调后的推出比较。

Extended Data Table 1 | Network parameters and detailed metrics show how our Swift system compares with other approaches

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a, Training hyperparameters. **b**, Comparison with baselines that are provided with the same observation noise model used by our approach. **c**, Evaluation in simulation, with idealized dynamics (top) versus realistic dynamics (bottom) and ground-truth observations (left) versus noisy observations (right). We report the fastest achieved collision-free lap time in seconds, the average and smallest gate margin of successfully passed gates and the percentage of track completed. We compare our approach with a learning-based approach that performs zero-shot transfer, with and without domain randomization during training, as well as a traditional planning and control approach[28]. **d**, Comparison of the average speed, power, thrust, time and distance travelled for each pilot during the fastest flown race. Best numbers are indicated in bold.

扩展数据表 1 | 网络参数和详细指标显示了我们的 Swift 系统与其他方法的比较

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a，训练超参数。b，与我们的方法使用的相同观察噪声模型提供的基线进行比较。c，模拟评估，理想化动力学（上）与现实动力学（下）以及真实观测值（左）与噪声观测值（右）。我们报告最快达到的无碰撞单圈时间（以秒为单位）、成功通过闸门的平均和最小闸门余量以及赛道完成的百分比。我们将我们的方法与执行零样本迁移的基于学习的方法进行比较，并且

训练期间无需域随机化，也无需传统的规划和控制方法[28]。d，在最快飞行比赛中每位飞行员的平均速度、功率、推力、时间和距离的比较。最佳数字以粗体表示。